

CalibiAI

Personalized Learning Engine

Complete Engineering Specification

Implementation-level build document for AI coding agents (Arena agent mode)

Stack: Next.js 15 · Supabase (DB + Auth) · Amazon Bedrock · JSON Knowledge Base

Scope: Personalized Learning Engine ONLY. Auth, Dashboard shell, Community, LMS and Recruiter modules already exist.

Version 1.0

Table of Contents

Table of Contents	2
How to Use This Document	7
Chapter 1 — Feature Overview	8
1.1 Purpose.....	8
1.2 Goals.....	8
1.3 User Journey.....	8
1.4 Business Logic.....	8
1.5 Success Metrics	9
1.6 Future Vision	9
Implementation Checklist (Mandatory)	9
Chapter 2 — Complete Onboarding Flow	11
2.1 Wizard Shell, Progress Bar & Animations	11
2.2 Step 1 — Welcome.....	11
2.3 Step 2 — Profile	11
2.4 Step 3 — Choose AI Career Role.....	11
2.5 Step 4 — Connect (Optional): GitHub & Resume.....	12
2.6 Step 5 — Ready / Start Assessment	12
2.7 Loading, Empty & Error States.....	13
2.8 Edge Cases	13
Implementation Checklist (Mandatory)	13
Chapter 3 — Assessment Engine	15
3.1 Question Bank & Data Contract	15
3.2 Question Loading & Randomization.....	15
3.3 Click-to-Reveal UX (Next.js)	15
3.4 Skill Categories, Difficulty & Weight	16
3.5 Scoring Algorithm (Deterministic, No AI).....	16
3.6 Knowledge Graph Generation	16
3.7 Weak & Strong Skill Detection	16
3.8 Level Determination (Beginner vs Intermediate)	17
3.9 Assessment Retry & History	17
3.10 Edge Cases	17
3.11 Performance.....	17
Implementation Checklist (Mandatory)	17
Chapter 4 — Roadmap Assignment Engine.....	19
4.1 Roadmap JSON Data Contract	19
4.2 File Selection (Deterministic).....	19
4.3 Beginner vs Intermediate	19

4.4 Assignment Procedure	19
4.5 How Bedrock Personalizes (Overlay, Not Mutation)	20
4.6 Prerequisites, Unlocking & Dependencies	20
4.7 Module Unlocking State Machine	20
4.8 Estimated Completion	20
4.9 Edge Cases	21
4.10 Performance	21
Implementation Checklist (Mandatory)	21
Chapter 5 — Dashboard Learning Experience	22
5.1 Layout	22
5.2 Widgets	22
5.3 Today's Mission (summary; full logic Ch.6)	22
5.4 Talent Score & Level (summary; full model Ch.8)	22
5.5 Completion Status & Current Module	22
5.6 User Experience	22
5.7 Aggregate Payload (GET /api/dashboard)	23
5.8 Edge Cases	23
5.9 Performance	23
Implementation Checklist (Mandatory)	23
Chapter 6 — Daily Mission Engine	25
6.1 How Today's Mission Is Generated	25
6.2 Task Types (from the day JSON)	25
6.3 Estimated Time	25
6.4 Completion Tracking	25
6.5 Missed Day Logic	26
6.6 Weekend Logic	26
6.7 UI / UX	26
6.8 Edge Cases	26
6.9 Performance	26
Implementation Checklist (Mandatory)	26
Chapter 7 — Progress Tracking Engine	28
7.1 What Is Tracked	28
7.2 Learning Time (Sessions)	28
7.3 Completion Percentage	28
7.4 Consistency & Streak	28
7.5 Weekly & Monthly Progress	28
7.6 GitHub Activity Sync	29
7.7 UI / UX	29

7.8 Edge Cases	29
7.9 Performance	29
Implementation Checklist (Mandatory)	29
Chapter 8 — Talent Score (Max 1000)	31
8.1 Component Weights (sum = 1000)	31
8.2 Component Formulas	31
8.3 XP vs Talent Score	31
8.4 Levels & Promotion Rules	31
8.5 Badges	32
8.6 Weekly Goals	32
8.7 Recompute Lifecycle	32
8.8 Edge Cases	32
8.9 Performance	32
Implementation Checklist (Mandatory)	32
Chapter 9 — Weekly Review Engine	34
9.1 Trigger & Scheduling	34
9.2 Input Assembled for the Agent	34
9.3 Agent Output (strict JSON)	34
9.4 Report Sections (UI)	34
9.5 Roadmap Update Merge	35
9.6 Edge Cases	35
9.7 Performance & Cost	35
Implementation Checklist (Mandatory)	35
Chapter 10 — Amazon Bedrock	37
10.1 Architecture	37
10.2 Prompt Design	37
10.3 Memory & Context	37
10.4 Knowledge Base Boundary	37
10.5 Caching	37
10.6 Retry, Fallback, Latency	38
10.7 Cost Optimization	38
10.8 Security	38
10.9 Bedrock Responsibilities (EXHAUSTIVE — nothing else)	38
Implementation Checklist (Mandatory)	38
Chapter 11 — AI Agents	40
11.1 Roadmap Agent	40
11.2 Assessment Analysis Agent (Advisory)	40
11.3 Daily Mission Agent	41

11.4 Weekly Review Agent	41
11.5 Talent Score Recommendation Agent.....	42
11.6 Learning Recommendation Agent	42
11.7 Project Review Agent.....	43
Implementation Checklist (Mandatory)	43
Chapter 12 — Supabase Database Design	45
12.1 Conventions	45
12.2 Core Tables	45
12.3 Assessment Tables	46
12.4 Roadmap Tables.....	46
12.5 Daily Mission & Task Tables	47
12.6 Projects.....	48
12.7 Tracking, Score & AI Tables.....	48
12.8 Relationships (summary)	50
12.9 Indexes	50
12.10 Triggers	51
12.11 RLS Policies.....	51
Implementation Checklist (Mandatory)	51
Chapter 13 — API Design	52
13.1 Conventions	52
13.2 Endpoint Catalog.....	52
13.3 Detailed Examples.....	53
13.4 Error Codes	53
13.5 Validation & Security	53
Implementation Checklist (Mandatory).....	53
Chapter 14 — Folder Structure.....	55
14.1 Complete Tree	55
14.2 Layering Rules	55
14.3 Naming & Conventions	56
Implementation Checklist (Mandatory)	56
Chapter 15 — JSON Standards.....	57
15.1 Assessment JSON	57
15.2 Roadmap JSON.....	57
15.3 Embedded Sub-Schemas (inside a day).....	57
15.4 Knowledge Graph JSON (engine-generated).....	58
15.5 Personalization Overlay JSON (engine + Roadmap Agent).....	58
15.6 Validation Rules (load-time).....	58
Implementation Checklist (Mandatory).....	58

Chapter 16 — Complete Sequence Diagram	59
16.1 Login -> Roadmap (first run)	59
16.2 Daily Learning Loop	59
16.3 Weekly Review Loop	59
16.4 Failure & Fallback Paths	59
16.5 End-to-End Invariants	60
Implementation Checklist (Mandatory)	60
16.6 Build Order for the Agent	60

How to Use This Document

This is the single source of truth for building the CalibiAI Personalized Learning Engine. An AI coding agent with access to the GitHub repository should implement the entire feature from this document alone, without asking clarifying questions.

Repository knowledge base (already committed): The curriculum content is already generated and lives in the repo. Read these paths directly:

- **content/roadmap/** — Eight 45-day roadmap JSON files (beginner + intermediate for each of four roles).
- **content/assessment/** — Four 100-question assessment banks (one per role).

Amazon Bedrock NEVER generates curriculum. It only personalizes ordering, recommendations, weekly reviews, daily motivation and project feedback. All questions, days, projects, quizzes, articles, videos and research papers are pre-authored in the JSON files.

Exact content file names in the repository

Role	Assessment file — content/assessment/	Roadmap files — content/roadmap/
GenAI Engineer	genai_engineer_beginner_assessment.json	roadmap_genai_beginner.json / roadmap_genai_intermediate.json
AI Engineer	ai_engineer_beginner_assessment.json	roadmap_ai_engineer_beginner.json / roadmap_ai_engineer_intermediate.json
AI Automation Engineer	ai_automation_engineer_beginner_assessment.json	roadmap_automation_beginner.json / roadmap_automation_intermediate.json
Data Science Engineer	data_science_engineer_beginner_assessment.json	roadmap_ds_beginner.json / roadmap_ds_intermediate.json

FILENAME NOTE: the Data Science intermediate roadmap is spelled roadmap_ds_intermediate.json (missing an 'e'). The role->file resolver MUST use this exact string. Do not rename repository files — map to them.

Every chapter ends with a mandatory 11-point Implementation Checklist. Treat each row as an acceptance gate.

Chapter 1 — Feature Overview

1.1 Purpose

The Personalized Learning Engine converts a newly-authenticated user into an active learner on a role-specific, difficulty-calibrated 45-day roadmap, and keeps them progressing daily. It owns everything between successful login and the recurring weekly AI review: onboarding, the placement assessment, skill scoring, knowledge-graph construction, roadmap assignment, daily missions, progress tracking, the Talent Score (max 1000), and the Bedrock-driven weekly review that re-sequences the roadmap.

It does NOT own authentication, the dashboard chrome/navigation shell, community, LMS or recruiter features. Those modules already exist and are consumed as read-only dependencies (for example, the engine reads an existing community score but never writes community data).

1.2 Goals

- **Zero-question onboarding** — Every student reaches a personalized roadmap in under 5 minutes with no ambiguity.
- **Accurate placement** — A 20-question placement test (drawn from a 100-question bank) reliably classifies the learner as Beginner or Intermediate.
- **Deterministic assignment** — Role + level deterministically selects one roadmap JSON; Bedrock only re-orders and annotates.
- **Daily habit loop** — A single 'Today's Mission' card drives watch -> read -> quiz -> assignment -> commit -> complete every day.
- **Measurable talent** — A transparent 1000-point Talent Score aggregates assessment, projects, assignments, learning, consistency, GitHub and community.
- **Adaptive feedback** — A weekly Bedrock review produces strengths, weaknesses, missed topics, and a re-sequenced roadmap.

1.3 User Journey

1. User logs in (existing Supabase Auth). Engine checks `profiles.onboarding_completed`.
2. If false: onboarding wizard (profile -> role -> optional GitHub/resume -> start assessment).
3. 20-question placement assessment (2 randomly selected per skill from the role's 100-question bank).
4. Backend computes skill scores (NO AI scoring) and persists `skill_scores` + a `knowledge_graph` row.
5. Level determination: Beginner or Intermediate from the aggregate weighted score.
6. Role + level selects the roadmap JSON; a `user_roadmaps` row is created and days are expanded into `roadmap_progress`.
7. Bedrock personalizes sequencing (skip / reorder / reinforce) and writes the personalization overlay.
8. User lands on the Dashboard: Today's Mission, roadmap %, streak, Talent Score, next project.
9. User completes daily tasks; `task_progress` + `learning_sessions` + `activity_logs` record everything.
10. Talent Score and XP update in real time; streak / consistency update at day boundaries.
11. Every 7 active days the Weekly Review Engine runs Bedrock and updates the personalized roadmap.

1.4 Business Logic

- **Placement, not certification** — The assessment has no pass/fail (`passing_score` = null). It positions the learner on the roadmap only.

- **Deterministic core, probabilistic polish** — Scoring, level and file selection are deterministic and reproducible. Bedrock adds an overlay that can always be discarded to fall back to raw JSON order.
- **Content immutability** — The engine never mutates content JSON. Personalization is stored as references / overrides in Supabase, keyed to day numbers and content ids.
- **One active roadmap per user per role** — Re-taking the assessment creates a new user_roadmaps version; history is preserved, never corrupted.

1.5 Success Metrics

Metric	Definition	Target
Onboarding completion	users reaching dashboard / users who start onboarding	>= 90%
Time-to-roadmap	login -> first dashboard render (p95)	< 5 min
D1 / D7 activation	users completing >=1 mission on day 1 / day 7	>= 60% / >= 35%
Weekly review delivery	eligible users getting a review within 24h	>= 99%
Assessment latency	submit -> score + graph persisted (p95)	< 1.5 s
Bedrock personalization	assignment -> overlay ready (p95, async)	< 8 s

1.6 Future Vision

- Per-skill micro-roadmaps and remedial reinforcement days auto-inserted when a skill regresses.
- Multi-role learners (switch role while retaining shared skill scores).
- An Advanced (expert) level tier beyond Intermediate.
- Real-time collaborative missions and peer accountability (consuming, not building, community).
- Bedrock-adjusted quiz difficulty within the fixed content bank.

Implementation Checklist (Mandatory)

Aspect	Implementation Detail
1. Frontend Components	OnboardingWizard, AssessmentRunner, DashboardHome (mission card mount), RoadmapTimeline — under app/(learning)/.
2. Backend Services	onboardingService, assessmentService, roadmapService, dashboardService, talentScoreService, weeklyReviewService (lib/services/).
3. Supabase Tables Used	profiles, roles, assessments, assessment_results, skill_scores, knowledge_graph, roadmaps, user_roadmaps, roadmap_progress, daily_tasks, task_progress, learning_sessions, talent_scores, weekly_reports, activity_logs.
4. APIs Required	POST /api/onboarding, POST /api/assessment/submit, POST /api/roadmap/assign, GET /api/dashboard, POST /api/weekly-review/run.
5. Amazon Bedrock Prompt	Only roadmap personalization, weekly review, daily motivation, project feedback, learning recommendations (Ch.10-11).
6. JSON Files Consumed	content/assessment/*.json, content/roadmap/*.json.
7. State Management	Server Components for reads; TanStack Query for mutations/optimistic UI; Zustand for the onboarding step machine.
8. Error Handling	Typed Result envelopes { data, error }; Bedrock failures degrade to raw JSON ordering; all writes idempotent by natural keys.
9. Loading States	Route-level Suspense + skeletons; per-step wizard spinners; streamed Bedrock overlay with a 'personalizing...' badge.
10. Testing Scenarios	E2E of full flow; unit tests for scoring determinism; JSON schema contract tests; Bedrock fallback simulation.
11. Future Scalability	Content on JSON+CDN; overlays are per-user rows; agents stateless and horizontally scalable.

Chapter 2 — Complete Onboarding Flow

Onboarding begins immediately after login when `profiles.onboarding_completed = false`. It is a 5-step wizard hosted at `/onboarding` as a client-driven flow over Server Actions. The wizard step machine lives in a Zustand store so a refresh restores the exact step from `profiles.onboarding_step`.

2.1 Wizard Shell, Progress Bar & Animations

- **Layout** — Centered card, max-width 640px, on a subtle gradient. Left-to-right step progress bar pinned to the top of the card.
- **Progress bar** — 5 segments (Welcome, Profile, Role, Connect, Ready). Filled segments use ACCENT; current segment pulses. Width = $(\text{currentStep}/5) * 100\%$.
- **Step transitions** — Framer Motion slide+fade (x: 24 -> 0, opacity 0 -> 1, 220ms ease-out). Back navigation reverses direction.
- **Persistence** — On each Next, a Server Action writes `profiles.onboarding_step`. Refresh rehydrates from the DB, not just `localStorage`.
- **Buttons** — Primary 'Continue' (disabled until step valid), secondary 'Back' (hidden on step 1), text 'Skip' where allowed. Enter key submits the active step.

2.2 Step 1 — Welcome

A one-screen welcome: brand, a single sentence ('Let's build your personalized 45-day path'), estimated time (~4 min), and a single 'Get started' button. No inputs. Pressing start writes `onboarding_step=2`.

2.3 Step 2 — Profile

Fields: `full_name` (text, required, 2-60 chars), `display_name` (text, optional, defaults to first token of `full_name`), `country` (select, required, ISO list), `timezone` (auto-detected via Intl, editable select), `education_level` (select: High School / Undergraduate / Graduate / Professional), `experience_level` (radio: New to tech / Some coding / Experienced), `weekly_hours` (slider 3-40, default 10).

Validation

- `full_name`: trim, reject empty/emoji-only, 2-60 chars. Error under field, red border, aria-invalid.
- `country` & `education_level` & `experience_level`: required; Continue disabled until all set.
- `weekly_hours`: integer 3-40; used later to size Today's Mission and estimate completion date.

Buttons & UX

Continue is disabled (50% opacity, cursor not-allowed) until the step is valid. Inline validation fires on blur, not on every keystroke, to avoid noise. A green check appears next to a field once valid.

2.4 Step 3 — Choose AI Career Role

Purpose: Selecting the role deterministically fixes which assessment bank and which roadmap family the learner receives.

UI

- Four large selectable cards (radio-group semantics), one per role, each with icon, title, one-line description and 'you'll learn' chips.
- Selected card gets an ACCENT border + check badge; only one selectable.

- A 'What does each role mean?' expander shows the skills list pulled from the assessment JSON `skills_assessed` array.

The four roles and their bound content

Role value (stored)	Assessment bank	Roadmap family
genai_engineer	genai_engineer_beginner_assessment.json	roadmap_genai_{level}.json
ai_engineer	ai_engineer_beginner_assessment.json	roadmap_ai_engineer_{level}.json
ai_automation_engineer	ai_automation_engineer_beginner_assessment.json	roadmap_automation_{level}.json
data_science_engineer	data_science_engineer_beginner_assessment.json	roadmap_ds_{level}.json*

*Data Science intermediate resolves to `roadmap_ds_intermediate.json` (repo spelling). Store role as a stable enum in `profiles.role` and `roles` table; never store the display label.

Validation & Edge Cases

- Exactly one role required. Continue disabled until selected.
- If a user returns and changes role before finishing onboarding, discard any in-progress `assessment_results` for the old role (`status='abandoned'`).
- Role cannot be changed silently after a roadmap is assigned; that path requires a role-switch flow (future).

2.5 Step 4 — Connect (Optional): GitHub & Resume

Purpose: Optional signals that enrich the Talent Score (GitHub) and let Bedrock tailor recommendations (resume). Fully skippable.

GitHub Flow

1. User clicks 'Connect GitHub'. We use Supabase Auth linked identity (OAuth) if available, else store a validated `github_username`.
2. On success store `profiles.github_username` and `github_connected=true`; show the avatar + public repo count as confirmation.
3. GitHub activity later feeds the GitHub sub-score (Ch.8) via a read-only daily sync job; onboarding only captures the handle.
4. If OAuth popup is blocked or cancelled: non-blocking toast 'You can connect GitHub later from Settings', GitHub stays disconnected.

Resume / LinkedIn (Optional)

- **Resume upload** — Accept `.pdf/.docx` <= 5MB. Store in Supabase Storage bucket `'resumes/{user_id}'`. Parse text server-side; persist extracted skills to `profiles.resume_skills` (jsonb).
- **LinkedIn** — Optional URL field, regex-validated (`linkedin.com/in/...`). Stored as `profiles.linkedin_url`; not parsed in v1.
- **Privacy** — A single sentence explains the resume is used only to personalize learning and is never shared with recruiters here.

Skip Logic

This entire step is optional. A prominent 'Skip for now' text button advances to Step 5 without writing any connect fields. Skipping is recorded as `activity_logs(action='onboarding_skip_connect')` for analytics, and does not affect placement.

2.6 Step 5 — Ready / Start Assessment

- Summary card recaps role, weekly hours and connected accounts.
- Explains the assessment: 20 questions, ~10 minutes, not graded pass/fail, used only to place you.
- Primary button 'Start Assessment' navigates to /assessment (Chapter 3). This does NOT set onboarding_completed yet.
- onboarding_completed flips to true only AFTER the roadmap is assigned (end of Ch.4), so an abandoned assessment re-enters here.

2.7 Loading, Empty & Error States

State	Trigger	UI
Saving step	Server Action in flight	Continue shows spinner, inputs disabled, optimistic advance on success
Role content load	Fetching skills_assessed from JSON	Skeleton chips for 300ms; cached after first read
GitHub connecting	OAuth round-trip	Button spinner + 'Connecting...'; 20s timeout -> retry toast
Resume parsing	Upload + parse	Progress bar; on failure keep file, show 'Couldn't read this file, you can skip'
Save failed	Network/DB error	Non-destructive banner, keep entered data, 'Retry' button

2.8 Edge Cases

- Refresh mid-wizard: rehydrate from profiles.onboarding_step; never lose entered fields (autosaved per step).
- User already completed onboarding but hits /onboarding: redirect to /dashboard.
- Duplicate submit (double-click Continue): idempotent Server Action keyed by (user_id, step).
- Timezone missing/blocked: default to UTC and let the user correct it.
- Slow network: steps remain interactive; only the in-flight write is blocked.
- Abandoned onboarding (leaves before Step 5): next login resumes at saved step.

Implementation Checklist (Mandatory)

Aspect	Implementation Detail
1. Frontend Components	OnboardingWizard, StepProgressBar, WelcomeStep, ProfileStep, RoleCardGroup, ConnectStep (GitHubButton, ResumeUploader), ReadyStep.
2. Backend Services	onboardingService.saveStep(), .setRole(), .connectGitHub(), .uploadResume(), .parseResume() (lib/services/onboarding).
3. Supabase Tables Used	profiles (onboarding_step, onboarding_completed, role, github_username, github_connected, linkedin_url, resume_skills, weekly_hours, timezone), roles, activity_logs.
4. APIs Required	POST /api/onboarding (step upsert), POST /api/onboarding/github, POST /api/onboarding/resume.
5. Amazon Bedrock Prompt	None in onboarding. (Resume skills are stored for later use by the Learning Recommendation Agent.)
6. JSON Files Consumed	content/assessment/{role}_beginner_assessment.json — read only for skills_assessed preview in the Role step.
7. State Management	Zustand wizard store (step, formDraft) mirrored to profiles.onboarding_step; TanStack Query mutations per step.
8. Error Handling	Field-level validation errors; non-blocking connect errors; idempotent step writes keyed by (user_id, step).
9. Loading States	Per-button spinners, skeleton role chips, resume progress bar, OAuth connecting state.

Aspect	Implementation Detail
10. Testing Scenarios	Validation matrix per field; refresh-resume; skip-connect path; role-change discards stale assessment; GitHub cancel path.
11. Future Scalability	Add roles by adding a roles row + content files; wizard is data-driven from the roles table, no code change per role.

Chapter 3 — Assessment Engine

The Assessment Engine renders a 20-question placement test, computes deterministic skill scores (no AI), builds a knowledge graph, classifies each skill and the learner overall, and persists everything for the Roadmap Assignment Engine. It consumes the role's 100-question bank in `content/assessment/`.

3.1 Question Bank & Data Contract

Each role bank contains exactly 100 questions = 10 skills × 10 questions (7 Easy + 3 Medium per skill, giving 70/30 overall). The top-level 'assessment' object carries selection rules; 'questions' is the pool.

```
// content/assessment/<role>_beginner_assessment.json
{
  "assessment": {
    "id": "genai-engineer-beginner-onboarding",
    "role": "GenAI Engineer",
    "type": "adaptive_placement_test",
    "total_questions": 100, "total_weight": 1000, "passing_score": null,
    "question_bank": true, "questions_per_skill": 10,
    "selection": { "strategy": "random", "questions_per_skill_selected": 2,
                  "generated_assessment_size": 20 },
    "difficulty_distribution": { "easy": 70, "medium": 30 },
    "difficulty_per_skill": { "easy": 7, "medium": 3 },
    "skills_assessed": ["Python", "Git", "...10 total..."],
    "answer_reveal": "on_select"
  },
  "questions": [ {
    "id": "Q1", "topic": "...", "skill": "Python", "difficulty": "Easy",
    "weight": 10, "question": "...", "options": ["a","b","c","d"],
    "correct_answer": 0, // 0-BASED index into options
    "explanation": "...", "roadmap_reference": "...", "estimated_time": "30 sec"
  } ]
}
```

CRITICAL: `correct_answer` is a 0-based index. The Next.js app compares the clicked option index to `correct_answer`, and reveals the explanation only after a click (`answer_reveal = 'on_select'`).

3.2 Question Loading & Randomization

1. Resolve the bank file from `profiles.role` (Ch.2 mapping table).
2. Group the 100 questions by skill (10 skills).
3. For each skill, randomly select exactly 2 questions -> 20 total. Preserve the per-skill 7E/3M shape loosely by preferring 1 Easy + 1 Medium when available, else 2 Easy.
4. Seed the RNG with a per-attempt seed stored in `assessment_results.seed` so an attempt is reproducible and cannot be silently reshuffled on refresh.
5. Shuffle the final 20 for presentation order; store the ordered question ids + seed in the attempt row.
6. Send ONLY id, topic, skill, difficulty, question, options to the client. NEVER send `correct_answer` or explanation until the option is selected (see 3.3).

3.3 Click-to-Reveal UX (Next.js)

- One question per screen. Progress indicator 'Question n of 20' + thin progress bar.
- User clicks an option. The client immediately calls `POST /api/assessment/answer` with `{ attempt_id, question_id, selected_index }`.
- The server validates, records the answer, and returns `{ correct_answer, is_correct, explanation }`.

- UI then colors the chosen option green if correct / red if wrong, always highlights the correct option green, and reveals the explanation card. 'Next' appears.
- This server-authoritative reveal prevents leaking answers in the initial payload and prevents client tampering with scores.
- Once answered, options lock (no changing the answer). 'Next' advances; last question shows 'Finish'.

3.4 Skill Categories, Difficulty & Weight

- **Skill** — One of the 10 skills_assessed for the role. Every question maps to exactly one skill.
- **Difficulty** — Easy or Medium. Used to weight correctness and to break ties in level classification.
- **Weight** — Base weight is 10 per question ($100 \times 10 = 1000$ max). Effective weight = base $\times$ difficulty multiplier (Easy 1.0, Medium 1.3), normalized per skill so each skill contributes equally to the 0-100 skill score.

3.5 Scoring Algorithm (Deterministic, No AI)

Runs server-side on Finish. Each of the 10 skills has 2 answered questions.

```
MULT = { Easy: 1.0, Medium: 1.3 }
for each skill s with its 2 questions:
    earned =  $\sum$  (is_correct ? MULT[difficulty] : 0)
    possible =  $\sum$  MULT[difficulty]
    skill_score[s] = round( $100 * \text{earned} / \text{possible}$ ) // 0..100
overall = round( mean(skill_score[s] for all 10 skills) ) // 0..100
// persist skill_score[] to skill_scores, overall to assessment_results.overall_score
```

Classification thresholds (per skill)

Skill score	Band	Meaning
0 – 39	weak	Needs full coverage; mark related days as reinforce/keep
40 – 74	developing	Standard coverage
75 – 100	strong	Candidate for skip/compress on beginner content

3.6 Knowledge Graph Generation

The knowledge graph is a JSON structure (stored in knowledge_graph.graph.jsonb) linking skills -> band -> roadmap day references. It is the bridge between assessment and roadmap personalization.

```
{
  "role": "genai_engineer", "level": "beginner", "overall": 58,
  "skills": [
    { "skill": "Python", "score": 82, "band": "strong",
      "related_days": [1,2,3], "recommendation": "compress" },
    { "skill": "RAG", "score": 25, "band": "weak",
      "related_days": [30,31,32], "recommendation": "reinforce" }
  ],
  "weak_skills": ["RAG", "AI Agents"],
  "strong_skills": ["Python", "Git"],
  "generated_at": "ISO8601"
}
```

related_days is derived by matching each question's roadmap_reference and skill to roadmap day skills_gained/title. This mapping is computed deterministically at scoring time so Bedrock receives a ready-made graph.

3.7 Weak & Strong Skill Detection

- weak_skills = skills with band='weak' (score < 40), ordered ascending by score.
- strong_skills = skills with band='strong' (score >= 75), ordered descending.

- Ties broken by difficulty: a skill answered right on a Medium ranks above one right only on Easy.
- These arrays drive: (a) level determination, (b) Bedrock reinforce/skip hints, (c) dashboard 'focus areas'.

3.8 Level Determination (Beginner vs Intermediate)

```
if overall >= 70 AND strong_skills.length >= 6 AND weak_skills.length <= 1:
    level = 'intermediate'
else:
    level = 'beginner'
// stored on assessment_results.level and user_roadmaps.level
```

Rationale: intermediate roadmaps assume broad baseline competence, so we require both a high mean AND breadth (few weak skills). Everyone else starts beginner, where strong skills are simply compressed rather than skipped.

3.9 Assessment Retry & History

- A user may retake the assessment (Settings -> 'Retake placement'). Each attempt is a new assessment_results row (attempt_number increments).
- Only the latest completed attempt is 'active' and drives the current roadmap; older attempts remain for history and trend charts.
- Cooldown: at most 1 retake per 24h to prevent gaming; enforced server-side.
- Retaking after a roadmap exists spawns a NEW user_roadmaps version; existing progress is archived, not deleted.
- Assessment history view: table of attempts with date, overall score, level, and per-skill sparkline.

3.10 Edge Cases

- Refresh mid-assessment: rehydrate from attempt row (ordered ids + recorded answers); resume at first unanswered.
- Bank has fewer than 2 questions for a skill (should never happen): fail closed with a schema-validation error at load, log to activity_logs.
- Duplicate answer submit: idempotent by (attempt_id, question_id); second call returns the stored result.
- User closes tab on last question: Finish is only committed server-side when all 20 are answered; otherwise status stays 'in_progress'.
- Tampered selected_index (out of 0..3): rejected with 422; recorded as invalid, not scored.
- All-correct or all-wrong: scoring still valid; level logic handles extremes.

3.11 Performance

- Bank JSON (~100 Q) loaded once per attempt, cached in memory / edge cache keyed by role+file-hash.
- Per-answer endpoint is a single-row upsert + one JSON lookup: p95 < 120ms.
- Finish scoring is O(20) arithmetic: p95 < 200ms; graph build O(skills×days): < 300ms.
- No Bedrock call in this chapter — placement is fully deterministic and fast.

Implementation Checklist (Mandatory)

Aspect	Implementation Detail
1. Frontend Components	AssessmentRunner, QuestionCard, OptionButton, ExplanationPanel, AssessmentProgress, AssessmentHistoryTable, SkillsSparkline.
2. Backend Services	assessmentService.startAttempt(), .recordAnswer(), .finishAttempt(), scoringEngine.computeSkillScores(), graphBuilder.build().

Aspect	Implementation Detail
3. Supabase Tables Used	assessments, assessment_results (attempt_number, seed, status, overall_score, level), skill_scores, knowledge_graph, activity_logs.
4. APIs Required	POST /api/assessment/start, POST /api/assessment/answer, POST /api/assessment/submit, GET /api/assessment/history.
5. Amazon Bedrock Prompt	None — scoring and classification are deterministic. Bedrock consumes the resulting knowledge_graph in Ch.4.
6. JSON Files Consumed	content/assessment/<role>_beginner_assessment.json (question pool + selection rules).
7. State Management	Attempt state server-authoritative; client holds only current index + revealed result; TanStack Query for answer mutations.
8. Error Handling	422 on tampered/invalid index; idempotent answers; schema validation on bank load; retry cooldown enforcement.
9. Loading States	Question skeleton on start; per-option pending state during reveal round-trip; finish spinner -> redirect to /roadmap/assign.
10. Testing Scenarios	Determinism (same seed+answers => same scores); reveal never leaks answers early; retry cooldown; refresh-resume; extreme scores.
11. Future Scalability	Banks are per-role files; adding intermediate assessment banks is additive; selection strategy is config in the JSON.

Chapter 4 — Roadmap Assignment Engine

This engine turns (role, level, knowledge_graph) into a live, per-user roadmap. Selection is deterministic; Bedrock adds a personalization overlay that reorders, compresses, or reinforces days without ever editing content.

4.1 Roadmap JSON Data Contract

```
// content/roadmap/roadmap_<family>_<level>.json
{
  "roadmap": { "title": "45-Day GenAI Engineer Roadmap", "role": "GenAI Engineer",
               "level": "Beginner", "total_days": 45, "provider": "CalibiAI",
               "description": "...", "outcome": "..." },
  "days": [ {
    "day": 1, "title": "...", "objectives": [...], "topics": [...],
    "estimated_time": "5-6 hours", "difficulty": "Beginner",
    "prerequisites": [...],
    "beginner_explanation": "...", // or intermediate_explanation for intermediate files
    "practical_task": "...", "mini_project": "...", "assignment": "...",
    "quiz": {...}, "youtube": [...], "official_docs": [...],
    "github_repositories": [...], "research_papers": [...],
    "skills_gained": [...], "expected_outcome": "..."
  } ]
}
```

4.2 File Selection (Deterministic)

```
const FAMILY = { genai_engineer:'genai', ai_engineer:'ai_engineer',
                 ai_automation_engineer:'automation', data_science_engineer:'ds' };
function roadmapFile(role, level){
  const fam = FAMILY[role];
  // NOTE: DS intermediate is misspelled in the repo
  if (fam==='ds' && level==='intermediate') return 'roadmap_ds_intermediate.json';
  return `roadmap_${fam}_${level}.json`;
}
```

Resolution is pure and unit-tested against the actual repo listing so a missing/misspelled file is caught in CI, not at runtime.

4.3 Beginner vs Intermediate

- **Beginner file** — Full 45 days with beginner_explanation. Strong skills are compressed (marked optional/fast-track), never removed.
- **Intermediate file** — 45 days with intermediate_explanation, denser topics, assumes baseline competence. Chosen only when Ch.3 level='intermediate'.
- **Why not skip days on beginner** — Beginners benefit from exposure even to known topics; we compress (shorter estimated time, 'review' badge) instead of hiding.

4.4 Assignment Procedure

1. Read level + knowledge_graph from the latest completed assessment_results.
2. Resolve roadmapFile(role, level) and load+validate the JSON against the day schema.
3. Ensure a roadmaps row exists for this file (upsert by file_key + version hash of the JSON).
4. Create a user_roadmaps row: { user_id, roadmap_id, role, level, status:'active', version, start_date }.
5. Expand all 45 days into roadmap_progress rows: { user_roadmap_id, day, status:'locked', unlock_at:null }.
6. Apply prerequisite/unlock rules (4.6) to set day 1 (and any no-prereq days) to 'unlocked'.

7. Fire the Roadmap Agent (Bedrock) asynchronously to produce the personalization overlay (4.5). The dashboard is usable immediately with raw order; overlay is merged when ready.
8. Set `profiles.onboarding_completed=true` and redirect to `/dashboard`.

4.5 How Bedrock Personalizes (Overlay, Not Mutation)

The Roadmap Agent receives the knowledge graph + the list of day summaries (day, title, skills_gained, difficulty). It returns a strict-JSON overlay of per-day actions. Content is never rewritten.

```
// Bedrock output -> user_roadmaps.personalization (jsonb)
{
  "sequence": [1,2,5,3,4, ...],           // reordered day numbers (must be a permutation)
  "day_actions": {
    "1": { "action": "compress", "reason": "Python strong (82)", "badge": "Review" },
    "30": { "action": "reinforce", "reason": "RAG weak (25)",
            "extra_focus": ["revisit embeddings basics"] },
    "12": { "action": "keep" }
  },
  "focus_skills": ["RAG", "AI Agents"],
  "generated_by": "roadmap_agent", "model": "...", "generated_at": "ISO8601"
}
```

- **compress** — Day stays but is flagged review/fast-track; `estimated_time` shown reduced; counts less toward daily load.
- **reinforce** — Day emphasized; `extra_focus` notes surfaced; may seed an extra practice item in the Daily Mission.
- **keep** — Default; standard presentation and order.
- **Validation of overlay** — sequence must be a permutation of 1..45; unknown actions rejected; on any violation the overlay is discarded and raw order is used.

4.6 Prerequisites, Unlocking & Dependencies

- Each day's prerequisites array is mapped to prior day numbers by matching skills/titles at assignment time -> `roadmap_progress.prereq_days (int[])`.
- A day unlocks when ALL `prereq_days` are `status='completed'` (or the day has no prereqs).
- Linear fallback: if prereqs can't be resolved, default to strictly sequential unlock (day N unlocks when N-1 completes).
- The personalization 'sequence' changes DISPLAY order and unlock order together, but never lets a day unlock before its prereqs regardless of sequence.
- Compressed days still must be 'completed' (a one-click 'mark reviewed') to unlock dependents — no silent skipping.

4.7 Module Unlocking State Machine

Status	Enter when	UI
locked	prereqs incomplete	Grayed, lock icon, tooltip lists blocking days
unlocked	all prereqs complete	Actionable, 'Start' button
in_progress	user opens/starts any task in the day	Progress ring, resume button
completed	all required tasks in the day done	Green check; unlocks dependents

4.8 Estimated Completion

```
totalHours = Σ day.estimated_time (parsed, compressed days at 0.4x)
perWeek    = profiles.weekly_hours
```

```

etaWeeks = ceil(totalHours / perWeek)
etaDate = start_date + etaWeeks*7 days // shown on dashboard & roadmap header

```

4.9 Edge Cases

- Roadmap file missing/invalid: block assignment, surface a clear error, log to activity_logs; do NOT flip onboarding_completed.
- Bedrock overlay times out/fails: assign with raw JSON order; retry overlay via Weekly Review; user is never blocked.
- JSON content version changes after assignment: existing user_roadmaps keep their pinned version hash; new users get the new version.
- User retakes assessment and changes level: archive current user_roadmaps (status='archived'), create a new active one; carry over completed-day credit where day numbers/skills match.
- Overlay 'sequence' not a permutation: discard overlay, keep raw order, log validation error.

4.10 Performance

- Assignment (select+expand 45 rows) is one transaction: p95 < 400ms.
- Overlay is async; dashboard renders with raw order immediately (no blocking on Bedrock).
- Roadmap JSON cached by file_key + hash; validated once per version.

Implementation Checklist (Mandatory)

Aspect	Implementation Detail
1. Frontend Components	RoadmapTimeline, DayCard, LockBadge, PersonalizationBadge (Review/Focus), EtaHeader, AssigningLoader.
2. Backend Services	roadmapService.selectFile(), .assign(), .expandDays(), .applyOverlay(); roadmapAgent (Bedrock) invoked async.
3. Supabase Tables Used	roadmaps, user_roadmaps (level, version, status, personalization jsonb, start_date), roadmap_progress (day, status, prereq_days, unlock_at), assessment_results, skill_scores, knowledge_graph, activity_logs.
4. APIs Required	POST /api/roadmap/assign, GET /api/roadmap/:userRoadmapId, POST /api/roadmap/overlay/rebuild.
5. Amazon Bedrock Prompt	Roadmap Agent: input=knowledge_graph + day summaries; output=strict-JSON overlay {sequence, day_actions, focus_skills} (Ch.11).
6. JSON Files Consumed	content/roadmap/roadmap_<family>_<level>.json (incl. roadmap_ds_intermediate.json).
7. State Management	Server Components read the assigned roadmap; overlay merge is server-side; client subscribes for the 'personalized' flip.
8. Error Handling	Missing/invalid file blocks assignment; overlay validation (permutation, known actions); async retry via weekly review.
9. Loading States	'Building your roadmap...' screen during assign; timeline shows raw order with a 'Personalizing...' chip until overlay lands.
10. Testing Scenarios	File resolver incl. DS misspelling; permutation validation; prereq unlock correctness; ETA math; retake/version archival.
11. Future Scalability	New roadmaps = new files + roadmaps rows; overlay per user row; version pinning isolates content changes.

Chapter 5 — Dashboard Learning Experience

The dashboard is the learner's daily home, rendered at /dashboard inside the existing shell (the engine mounts widgets; it does not own the nav chrome). It is a Server Component that reads a single aggregated payload from GET /api/dashboard and hydrates interactive widgets.

5.1 Layout

- **Primary column** — Today's Mission card (hero), then Current Module, then Next Recommendation.
- **Secondary column** — Talent Score dial, Roadmap Progress ring, Weekly Progress, Streak, Learning Calendar, Current Project.
- **Responsive** — Two columns $\geq 1024\text{px}$; single stacked column on mobile with Today's Mission always first.

5.2 Widgets

Widget	Data source	Shows
Today's Mission	daily_tasks for today (Ch.6)	Ordered tasks (video/article/quiz/assignment/commit) + total est. time
Roadmap Progress	roadmap_progress	completed_days / 45, % ring, ETA date
Weekly Progress	task_progress + learning_sessions	this week's tasks done, minutes learned, goal bar
Learning Calendar	activity_logs (daily rollup)	GitHub-style heatmap of active days (last 12 weeks)
Daily XP	task_progress.xp	XP earned today + streak multiplier
Talent Score	talent_scores	current /1000, level, delta vs last week
Current Module	roadmap_progress (in_progress)	active day title, objectives, resume button
Current Project	roadmap day.mini_project	active project + submit/commit CTA
Next Recommendation	Recommendation Agent / next unlocked day	one suggested next action

5.3 Today's Mission (summary; full logic Ch.6)

- Shows the current unlocked day's tasks as a checklist with per-item est. time and type icon.
- A single 'Start today's mission' CTA opens the first incomplete task.
- Header shows day number, streak flame, and 'X min today' based on profiles.weekly_hours.

5.4 Talent Score & Level (summary; full model Ch.8)

- Circular dial 0-1000 with level label (e.g. 'Explorer', 'Builder', 'Specialist').
- Tap opens a breakdown drawer: assessment, projects, assignments, learning, consistency, GitHub, community, weekly goals.
- Delta chip (+/- since last weekly review) with color.

5.5 Completion Status & Current Module

- Roadmap ring = completed_days/45. Hover shows compressed vs full day counts.
- Current Module card resumes the in_progress day, or promotes the next unlocked day if none in progress.
- If all days complete: show a 'Roadmap complete' celebration + suggest intermediate roadmap / next role.

5.6 User Experience

- First paint <1s using cached aggregate; widgets stream in with skeletons.
- Optimistic completion: checking a task updates XP/streak instantly, reconciles with server.
- Empty states: no mission yet (overlay still building) -> friendly 'Preparing your first mission...'.
- Micro-interactions: XP counter tween, streak flame pulse on new day, confetti on day completion.

5.7 Aggregate Payload (GET /api/dashboard)

```
{
  "profile": { "display_name", "role", "level", "weekly_hours" },
  "talent": { "score", "level", "delta", "breakdown": {...} },
  "roadmap": { "completed_days", "total_days":45, "pct", "eta_date",
    "current_day": {...}, "next_day": {...} },
  "today": { "day", "tasks":[{" id,type,title,est_min,status }], "total_min" },
  "week": { "tasks_done", "tasks_goal", "minutes", "minutes_goal" },
  "streak": { "current", "longest", "multiplier" },
  "calendar": [{ "date", "count" }],
  "recommendation": { "type", "title", "reason" }
}
```

5.8 Edge Cases

- Overlay not ready: render raw-order mission, show 'Personalizing...' chip; never block.
- Timezone at day boundary: 'today' computed in profiles.timezone; a user just past midnight sees the new day.
- No activity in >N days: show a gentle re-engagement banner (not punitive).
- Roadmap archived (retake in progress): show the active one only.

5.9 Performance

- Single aggregate endpoint (no widget waterfalls); cached per user for 60s, invalidated on task completion.
- Calendar rollup precomputed daily; not recomputed per request.
- Server Components + partial hydration keep JS payload small.

Implementation Checklist (Mandatory)

Aspect	Implementation Detail
1. Frontend Components	DashboardHome, TodaysMissionCard, TalentScoreDial, RoadmapProgressRing, WeeklyProgress, LearningCalendarHeatmap, DailyXpBadge, CurrentModuleCard, CurrentProjectCard, NextRecommendationCard.
2. Backend Services	dashboardService.getAggregate(); reads across progress/talent/session services; 60s cache.
3. Supabase Tables Used	profiles, user_roadmaps, roadmap_progress, daily_tasks, task_progress, learning_sessions, talent_scores, activity_logs, weekly_reports.
4. APIs Required	GET /api/dashboard (aggregate), POST /api/tasks/:id/complete (optimistic), GET /api/calendar.
5. Amazon Bedrock Prompt	Consumes Next Recommendation from the Recommendation Agent (cached daily). No synchronous Bedrock call on dashboard load.
6. JSON Files Consumed	content/roadmap/*.json (day details for current module/project rendering).
7. State Management	Server Components for reads; TanStack Query mutations with optimistic updates for task completion; Zustand for drawer UI.
8. Error Handling	Partial widget failure isolates to that card ('couldn't load, retry'); aggregate never 500s the whole page.
9. Loading States	Per-widget skeletons; streamed hydration; 'Preparing your first mission...' empty state.

Aspect	Implementation Detail
10. Testing Scenarios	Aggregate contract test; optimistic completion reconciliation; timezone day-boundary; overlay-not-ready path.
11. Future Scalability	Aggregate cache + daily rollups keep reads $O(1)$; widgets are independently cacheable and lazy.

Chapter 6 — Daily Mission Engine

The Daily Mission Engine turns the current unlocked roadmap day into a concrete, time-boxed checklist of tasks for 'today', tracks completion, and handles missed days and weekends. It writes `daily_tasks` + `task_progress` and is the primary driver of XP, streak and consistency.

6.1 How Today's Mission Is Generated

1. Determine 'today' in `profiles.timezone`. Find the active `user_roadmap` and its current day = first day with status in (unlocked, `in_progress`).
2. If a `daily_tasks` set already exists for (user, day, date) return it (idempotent). Otherwise generate it.
3. Read the roadmap day object and decompose its fields into ordered tasks (6.2).
4. Apply the personalization `day_action`: compressed days emit fewer/lighter tasks; reinforced days append an extra practice task from `extra_focus`.
5. Time-box to `profiles.weekly_hours` / active-days-per-week: if the day's `estimated_time` exceeds the daily budget, split across consecutive days (task carryover), keeping order.
6. Persist `daily_tasks` rows and return the ordered list to the dashboard.

6.2 Task Types (from the day JSON)

Task type	Source field	Completion signal
video	<code>youtube[]</code>	watched toggle (≥ 1 video) or time threshold
article	<code>official_docs[]</code> / <code>articles</code>	opened + 'mark read'
quiz	<code>quiz{}</code>	<code>quiz_attempts</code> score recorded ($\geq$ pass threshold or attempt made)
assignment	<code>assignment</code>	self-marked done + optional artifact link
practical_task	<code>practical_task</code>	self-marked done
project	<code>mini_project</code>	submission/commit link (spans multiple days)
research	<code>research_papers[]</code>	optional 'read' toggle (bonus XP)
commit	<code>github_repositories[]</code> / user repo	GitHub commit detected by daily sync (bonus)

Required-for-completion set = video + article + quiz + assignment/practical_task. project/research/commit are enriching but not blocking day completion (project has its own submission lifecycle).

6.3 Estimated Time

- Each task carries `est_min` derived from the day `estimated_time` split proportionally by type (video 25%, article 20%, quiz 10%, assignment/practical 45%).
- Compressed (review) days scale `est_min` by 0.4x.
- The mission header sums `est_min` into 'X min today'.

6.4 Completion Tracking

- Each task -> `task_progress` row { `user_id`, `daily_task_id`, `status`, `completed_at`, `xp`, `artifact_url` }.
- Completing a task: optimistic UI check -> `POST /api/tasks/:id/complete` -> awards XP (Ch.8) and appends `activity_logs`.
- A day is 'completed' when all required tasks are done; this flips `roadmap_progress.status='completed'` and evaluates dependents' unlock.
- Completing the day triggers a streak increment (once per calendar day) and a `talent_scores` recompute.

6.5 Missed Day Logic

- A 'missed day' = an active day with zero completed required tasks by the user's local midnight.
- Missing a day does NOT delete progress and does NOT lock content; it breaks the streak (streak.current -> 0) and reduces the consistency sub-score.
- Carryover: incomplete tasks roll into the next active day (kept at top). The roadmap does not silently advance past unfinished required tasks.
- After 3 consecutive missed days, flag for a re-engagement nudge and let the Weekly Review soften the plan (fewer tasks/day).

6.6 Weekend Logic

- profiles has learning_days (default Mon-Fri). Weekend (or non-learning) days are 'rest days': no mission generated, streak preserved (not broken), consistency unaffected.
- Users can opt into 7-day learning; then weekends generate missions normally.
- 'Catch-up' mode: a user may pull the next day's mission on a rest day voluntarily (counts as bonus, cannot be forced).

6.7 UI / UX

- Checklist with type icons, est_min, and a running 'X of Y done' + minutes bar.
- Tapping a task opens its player/panel (video embed, article reader, quiz runner, assignment form).
- Confetti + XP tween on day completion; streak flame animates on increment.
- Locked-next-day preview at the bottom to motivate ('Tomorrow: ...').

6.8 Edge Cases

- Timezone change/travel: 'today' recomputed from current profiles.timezone; no double-award (idempotent per date).
- Duplicate completion: idempotent by task_progress unique (user, daily_task_id).
- Day estimated_time unparseable: default 60 min, log warning.
- Overlay reinforced day with empty extra_focus: skip the extra task gracefully.
- User completes all 45 days early: mission engine shows 'roadmap complete' state, offers next steps.

6.9 Performance

- Mission generation is O(tasks-per-day) and cached per (user, date); regenerated only on personalization change.
- Completion endpoint: single upsert + XP recompute (bounded), p95 < 200ms.

Implementation Checklist (Mandatory)

Aspect	Implementation Detail
1. Frontend Components	TodaysMissionCard, TaskChecklist, TaskItem, VideoPlayerPanel, ArticleReader, QuizRunner, AssignmentForm, DayCompleteCelebration.
2. Backend Services	missionService.generateForDay(), .completeTask(), .rolloverIncomplete(); consumes roadmapService + talentScoreService.
3. Supabase Tables Used	daily_tasks, task_progress, roadmap_progress, quiz_attempts, learning_sessions, activity_logs, profiles(learning_days, weekly_hours, timezone).
4. APIs Required	GET /api/mission/today, POST /api/tasks/:id/complete, POST /api/mission/rollover.

Aspect	Implementation Detail
5. Amazon Bedrock Prompt	Daily Mission Agent may add a one-line motivation + (on reinforce days) suggest an extra focus task; failure => omit, never block.
6. JSON Files Consumed	content/roadmap/*.json (day fields: youtube, official_docs, quiz, assignment, practical_task, mini_project, research_papers).
7. State Management	TanStack Query for mission + optimistic task completion; Zustand for the active-task panel.
8. Error Handling	Idempotent completion; unparseable time fallback; missed-day handled without data loss; empty extra_focus safe.
9. Loading States	Checklist skeleton; per-task panel spinners; optimistic check with rollback on error.
10. Testing Scenarios	Idempotency; missed-day streak break; weekend rest-day preservation; carryover ordering; timezone boundary.
11. Future Scalability	Missions are per (user, date) rows; generation is pure from day JSON + overlay; horizontally scalable.

Chapter 7 — Progress Tracking Engine

This engine is the system of record for everything a learner does: videos watched, articles read, assignments and projects completed, quiz attempts, GitHub activity, learning time, completion %, and consistency. It feeds the dashboard, the Talent Score, and the Weekly Review. All events are append-only in activity_logs and rolled up into typed tables.

7.1 What Is Tracked

Signal	Captured by	Stored in
Video watched	player 'watched'/threshold event	task_progress + activity_logs(video_watch)
Article read	'mark read' / reader dwell	task_progress + activity_logs(article_read)
Assignment done	assignment form submit	task_progress(artifact_url)
Project submitted	commit/submission link	task_progress(project) + activity_logs(project_submit)
Quiz attempt	QuizRunner submit	quiz_attempts(score, answers) + activity_logs
GitHub activity	daily GitHub sync job	activity_logs(github_commit) rollup
Learning time	session start/heartbeat/stop	learning_sessions(duration_sec)
Completion %	derived from roadmap_progress	computed (not stored raw)
Consistency	day-boundary job over active days	talent_scores.consistency + streak

7.2 Learning Time (Sessions)

- A learning_sessions row opens when a task panel opens; heartbeats every 30s; closes on blur/close/idle(>5min).
- duration_sec accumulates only active time (idle excluded) to keep 'minutes learned' honest.
- Daily/weekly/monthly minutes are SUM over sessions; used for the weekly minutes goal and the learning sub-score.

7.3 Completion Percentage

```
roadmap_pct = completed_days / total_days(45)
day_pct     = completed_required_tasks / required_tasks_in_day
week_pct    = tasks_done_this_week / tasks_planned_this_week
// compressed days count as fractional load (0.4) for planned-hours, but 1 for day count
```

7.4 Consistency & Streak

- streak.current increments once per active day with >=1 completed required task; resets to 0 on a missed active day (weekends excluded unless 7-day mode).
- streak.longest tracks the max.
- consistency (0-100) = active_days_with_activity / expected_active_days over a trailing 4-week window.
- Both are recomputed by a day-boundary job (per timezone bucket) and on each completion.

7.5 Weekly & Monthly Progress

- **Weekly** — tasks done, minutes learned, days active, XP earned, quiz average — the payload the Weekly Review Agent consumes.
- **Monthly** — trend of weekly rollups; surfaced as charts (minutes/week, completion/week, talent delta/week).

- **Rollups** — materialized nightly into progress_rollups (or computed with cached aggregates) to keep dashboard reads $O(1)$.

7.6 GitHub Activity Sync

1. If profiles.github_connected, a daily job fetches public events for github_username.
2. Count commits/PRs in repos relevant to the roadmap (matched loosely by language/topic).
3. Write activity_logs(github_commit, count, date). Feeds the GitHub sub-score (Ch.8) and the calendar heatmap.
4. Read-only: the engine never writes to GitHub. Rate-limit aware with backoff; failures are non-fatal.

7.7 UI / UX

- Progress page: roadmap ring, weekly bars, minutes chart, streak calendar, quiz-average trend.
- Every widget links to the underlying day/task for drill-down.
- All numbers update optimistically on completion and reconcile with the nightly rollup.

7.8 Edge Cases

- Clock skew / offline: server timestamps are authoritative; client times are hints only.
- Session never closed (crash): capped at a max session length; idle heartbeats prevent runaway durations.
- GitHub handle invalid/renamed: mark github_connected=false, notify, stop syncing.
- Duplicate events: activity_logs dedup by (user, action, target, date) where applicable.
- Backfill on retake: archived roadmap progress excluded from current rollups but kept for history.

7.9 Performance

- Append-only writes are cheap; heavy reads served from nightly rollups + 60s dashboard cache.
- GitHub sync batched per user once daily; never on the request path.
- Indexes on (user_id, date) across activity_logs, learning_sessions, task_progress.

Implementation Checklist (Mandatory)

Aspect	Implementation Detail
1. Frontend Components	ProgressPage, RoadmapRing, WeeklyBars, MinutesChart, StreakCalendar, QuizTrend, DrilldownDrawer.
2. Backend Services	progressService.recordEvent(), .rollupNightly(), sessionService.heartbeat(), githubSyncJob.
3. Supabase Tables Used	activity_logs, learning_sessions, task_progress, quiz_attempts, roadmap_progress, progress_rollups, talent_scores, profiles.
4. APIs Required	POST /api/session/heartbeat, POST /api/session/close, GET /api/progress/weekly, GET /api/progress/monthly, POST /api/github/sync (internal).
5. Amazon Bedrock Prompt	None directly; rollups become the Weekly Review Agent input (Ch.9/11).
6. JSON Files Consumed	content/roadmap/*.json (to know required tasks per day for completion math).
7. State Management	Server reads from rollups; TanStack Query for live counters; session heartbeat via a small client interval.
8. Error Handling	Server-authoritative timestamps; session length caps; GitHub sync backoff + non-fatal; event dedup.
9. Loading States	Chart skeletons; optimistic counter updates; nightly reconcile.
10. Testing Scenarios	Idle exclusion in duration; streak reset on miss; weekend preservation; GitHub sync failure; rollup correctness vs raw logs.

Aspect	Implementation Detail
11. Future Scalability	Append-only + nightly rollups + per-(user,date) indexes scale linearly; GitHub sync sharded by user bucket.

Chapter 8 — Talent Score (Max 1000)

The Talent Score is a transparent 0-1000 composite that summarizes a learner's demonstrated ability and habits. It is deterministic, recomputed on every qualifying event and at each day boundary, and stored (with a full component breakdown) in `talent_scores`. Community is CONSUMED as an existing score; this engine never redesigns or writes community data.

8.1 Component Weights (sum = 1000)

Component	Max points	Basis
Assessment	150	Placement overall_score (0-100) scaled ×1.5
Projects	220	Completed mini/capstone projects, quality-weighted (Project Review Agent 0-100)
Assignments	150	Completed assignments across days (count vs available)
Learning	150	Minutes learned vs weekly goals (rolling, capped)
Consistency	150	Streak + trailing 4-week active-day ratio
GitHub	80	Synced commits/PRs on relevant repos (capped, diminishing returns)
Community	50	EXISTING community score, read-only, normalized to 0-50
Weekly Goals	50	Weekly goal completion (tasks + minutes)
TOTAL	1000	

8.2 Component Formulas

```

assessment = round(1.5 * overall_score) // 0..150
projects   = round(220 * Σ(projQuality/100) / projAvailable) // 0..220
assignments = round(150 * assignmentsDone / assignmentsAvail) // 0..150
learning   = round(150 * min(1, minutes / minutesGoalCum)) // 0..150
consistency = round(150 * (0.5*streakFactor + 0.5*activeRatio)) // 0..150
github     = round(80 * min(1, log1p(commits)/log1p(CAP))) // 0..80
community  = round(50 * existingCommunityScoreNorm) // 0..50 (read-only)
weeklyGoals = round(50 * goalsMetRatio) // 0..50
talent      = clamp(0, 1000, Σ all components)
streakFactor = min(1, streak.current / 30)
activeRatio  = activeDays / expectedActiveDays (trailing 4 weeks)

```

8.3 XP vs Talent Score

- **XP** — A fast, always-increasing motivational currency earned per task (video 10, article 10, quiz 20, assignment 30, project 100, commit 15), multiplied by the streak multiplier. Never decreases.
- **Talent Score** — A normalized 0-1000 snapshot of current ability/habits; can go down (e.g. consistency drops). XP fuels levels; Talent Score reflects standing.
- **Streak multiplier** — 1.0 base, +0.1 per 7-day streak tier, capped at 2.0. Applies to XP only.

8.4 Levels & Promotion Rules

Level	Talent range	Promotion rule
Explorer	0 – 249	Default start
Builder	250 – 499	Reach 250 AND complete ≥ 5 days
Practitioner	500 – 749	Reach 500 AND ≥ 2 projects completed
Specialist	750 – 899	Reach 750 AND consistency ≥ 70
Expert	900 – 1000	Reach 900 AND roadmap ≥ 90% AND ≥ 1 capstone

Promotions are evaluated on recompute; a promotion emits `activity_logs(level_up)` and a celebratory toast. Demotion is silent (no negative toast) — the dial simply reflects the lower band.

8.5 Badges

- Badges are milestone flags stored in `badges (user_id, code, earned_at)`. Examples: `first_day`, `week_warrior` (7-day streak), `quiz_ace` (5 quizzes $\geq 80\%$), `project_shipper` (first project), `night_owl`, `comeback` (return after 3+ missed days).
- Badges give one-time XP bonuses (not Talent points) and appear on the profile.
- Badge evaluation runs in the same recompute pass; idempotent by (user, code).

8.6 Weekly Goals

- Each week the engine sets `tasks_goal` and `minutes_goal` from `profiles.weekly_hours` and remaining roadmap load.
- $\text{goalsMetRatio} = 0.5 * \min(1, \text{tasksDone}/\text{tasksGoal}) + 0.5 * \min(1, \text{minutes}/\text{minutesGoal})$.
- Feeds both the `weeklyGoals` component (max 50) and the Weekly Review narrative.

8.7 Recompute Lifecycle

1. Triggered on: task completion, quiz submit, project review, GitHub sync, day boundary, weekly review.
2. Load current inputs (assessment, projects, assignments, minutes, streak, github, community, goals).
3. Compute components (8.2), sum, clamp, evaluate level + badges.
4. Write a NEW `talent_scores` row (append versioned) with component breakdown + delta vs previous.
5. Emit `activity_logs`; invalidate the dashboard cache.

8.8 Edge Cases

- No projects available yet: `projAvailable=0` -> projects component = 0 (not NaN); guard divisions.
- Community module offline: use last-known community score or 0; never block recompute.
- GitHub disconnected: `github` component = 0; no penalty beyond the missing points.
- Retake changes assessment: assessment component updates from the new active attempt.
- Clock/timezone: consistency uses server day buckets in the user's tz.

8.9 Performance

- Recompute reads pre-aggregated rollups (Ch.7); pure arithmetic; p95 < 150ms.
- Append-versioned rows make history/trend charts cheap and auditable.

Implementation Checklist (Mandatory)

Aspect	Implementation Detail
1. Frontend Components	TalentScoreDial, ScoreBreakdownDrawer, LevelBadge, XpBadge, BadgeShelf, WeeklyGoalBar, LevelUpToast.
2. Backend Services	<code>talentScoreService.recompute()</code> , <code>.evaluateLevel()</code> , <code>.evaluateBadges()</code> ; reads <code>progressService</code> rollups + <code>communityAdapter(read-only)</code> .
3. Supabase Tables Used	<code>talent_scores</code> (component breakdown, delta, level), <code>badges</code> , <code>weekly_goals</code> , <code>assessment_results</code> , <code>task_progress</code> , <code>learning_sessions</code> , <code>activity_logs</code> .
4. APIs Required	GET <code>/api/talent/current</code> , GET <code>/api/talent/history</code> , GET <code>/api/badges</code> , POST <code>/api/talent/recompute</code> (internal).

Aspect	Implementation Detail
5. Amazon Bedrock Prompt	Talent Score Recommendation Agent suggests HOW to raise the score (advisory text only; does not change the number).
6. JSON Files Consumed	content/roadmap/*.json (to count available assignments/projects per role for denominators).
7. State Management	Server-computed; dashboard reads latest row; TanStack Query invalidation on completion events.
8. Error Handling	Division guards; community/GitHub absence => 0 not error; idempotent badge/level evaluation.
9. Loading States	Dial shimmer; breakdown drawer skeleton; level-up celebration only on genuine promotion.
10. Testing Scenarios	Formula unit tests incl. boundaries; promotion/demotion; badge idempotency; community-offline; retake update.
11. Future Scalability	Append-versioned rows + rollup inputs; recompute is $O(1)$ in inputs; community read via adapter interface.

Chapter 9 — Weekly Review Engine

Every 7 active learning days, the Weekly Review Engine assembles a structured week summary, invokes the Weekly Review Agent (Bedrock) to produce narrative feedback and roadmap adjustments, persists a weekly_reports row, and updates the personalized roadmap overlay. It is the engine's main adaptive loop.

9.1 Trigger & Scheduling

- Eligibility: ≥ 7 active days since last review (or 7 calendar days if the user is active). Tracked via `user_roadmaps.last_review_at`.
- A scheduled job (per timezone bucket) scans eligible users and enqueues reviews; also runnable on-demand via `POST /api/weekly-review/run`.
- Idempotent per (user, week_index): a week is reviewed once; re-runs update the same row.
- If Bedrock is unavailable, generate a deterministic 'lite' review (stats only) and mark `ai_enriched=false`; retry enrichment later.

9.2 Input Assembled for the Agent

```
{
  "week_index": 3,
  "role": "genai_engineer", "level": "beginner",
  "stats": { "tasks_done": 18, "tasks_planned": 22, "minutes": 320,
    "days_active": 5, "quiz_avg": 76, "streak": 5 },
  "skills": { "weak": ["RAG"], "strong": ["Python", "Git"],
    "recent_quiz_by_skill": {...} },
  "roadmap": { "completed_days": 12, "current_day": 13, "focus_skills": ["RAG"] },
  "missed_topics": ["day 11 assignment"],
  "talent": { "score": 540, "delta": +35 }
}
```

9.3 Agent Output (strict JSON)

```
{
  "summary": "2-3 sentence recap",
  "strengths": [...],
  "weaknesses": [...],
  "recommendations": [ { "text": "...", "skill": "RAG", "action": "reinforce" } ],
  "missed_topics": [...],
  "suggested_projects": [...], // must reference existing day mini_projects
  "roadmap_updates": { "reinforce_days": [30,31], "compress_days": [14],
    "resequence": [ ... optional permutation ... ] },
  "motivation": "1-2 sentence encouragement"
}
```

All `roadmap_updates` are validated against the assigned roadmap (valid day numbers, permutation if resequence present) before being merged into `user_roadmaps.personalization`. Invalid suggestions are dropped; valid ones update the overlay.

9.4 Report Sections (UI)

- **Summary** — Human recap of the week + talent delta chip.
- **Strengths / Weaknesses** — Two columns, skill-tagged.
- **Recommendations** — Actionable cards; each can 'apply to roadmap' (already merged) or 'add to today'.
- **Missed Topics** — Links back to the exact day/task to recover.
- **Suggested Projects** — From existing day mini_projects only (never invented).
- **Motivation** — Short encouraging note (Daily Motivation shares this tone).

9.5 Roadmap Update Merge

1. Take valid roadmap_updates from the agent.
2. Merge into the existing personalization overlay: reinforce/compress flags updated, resequence applied only if a valid permutation and it respects prerequisites.
3. Bump user_roadmaps.personalization.version and set last_review_at.
4. Regenerate today's mission if the current day's action changed.
5. Never delete completed progress; only future/locked days are affected.

9.6 Edge Cases

- Very low activity week: still produce a supportive review, soften next week's plan (fewer tasks/day), no shaming tone.
- Perfect week: celebrate, offer optional stretch (extra research_papers / harder project).
- Bedrock returns malformed JSON: run through the repair/validate step (Ch.10); if still invalid, fall back to lite review.
- User retakes assessment mid-week: next review uses the new active attempt + roadmap.
- Multiple eligible weeks queued (user was away): review only the most recent week; summarize the gap.

9.7 Performance & Cost

- One Bedrock call per user per week; inputs are pre-aggregated (small prompt) to control tokens.
- Results cached in weekly_reports; UI reads the row, not Bedrock.
- Batch/queue reviews off the request path; p95 user-visible latency = row read (<100ms).

Implementation Checklist (Mandatory)

Aspect	Implementation Detail
1. Frontend Components	WeeklyReviewPage, ReviewSummaryCard, StrengthsWeaknessesColumns, RecommendationCard, MissedTopicsList, SuggestedProjects, MotivationBanner, ReviewHistory.
2. Backend Services	weeklyReviewService.assembleInput(), .run(), .mergeRoadmapUpdates(); weeklyReviewAgent (Bedrock); scheduler job.
3. Supabase Tables Used	weekly_reports (input, output, ai_enriched, week_index), user_roadmaps (personalization, last_review_at), roadmap_progress, talent_scores, activity_logs.
4. APIs Required	POST /api/weekly-review/run, GET /api/weekly-review/latest, GET /api/weekly-review/history.
5. Amazon Bedrock Prompt	Weekly Review Agent (Ch.11): structured input -> strict-JSON review + validated roadmap_updates.
6. JSON Files Consumed	content/roadmap/*.json (to validate suggested_projects & day numbers against real content).
7. State Management	Server-generated; UI reads weekly_reports; TanStack Query for history; recommendations 'add to today' mutate daily_tasks.
8. Error Handling	JSON repair+validate; lite fallback (ai_enriched=false); invalid roadmap_updates dropped; idempotent per week.
9. Loading States	'Generating your weekly review...' state when run on-demand; history list skeletons.
10. Testing Scenarios	Eligibility timing; idempotency per week; malformed-JSON fallback; roadmap_updates validation; low/perfect week tones.
11. Future Scalability	Queued off-request; 1 call/user/week; small prompts; reports stored for cheap reads and trend history.

Chapter 10 — Amazon Bedrock

Bedrock is the ONLY generative component. It personalizes; it never authors curriculum. It is invoked through a single internal gateway (lib/ai/bedrockClient) that standardizes model selection, prompting, JSON validation, caching, retries, fallback, cost and security. Every agent in Chapter 11 calls Bedrock through this gateway.

10.1 Architecture

- **Gateway** — One module wraps the AWS SDK BedrockRuntime (InvokeModel / Converse). No agent talks to AWS directly.
- **Model** — A configured Anthropic Claude model on Bedrock (model id in env BEDROCK_MODEL_ID). Region in AWS_REGION.
- **Invocation path** — Server-only (Next.js Route Handlers / Server Actions). Never called from the browser; AWS creds never reach the client.
- **Async by default** — Personalization and reviews run off the request path (queue/background) so the UI never blocks on Bedrock.
- **Determinism knobs** — temperature≤0.3 for structured tasks; response_format enforced via prompt + JSON schema validation.

```
// lib/ai/bedrockClient.ts (shape)
invoke({ agent, system, messages, schema, cacheKey, maxTokens, temperature }):
  1. if cache.has(cacheKey) -> return cached
  2. build request (system + messages), call Bedrock Converse
  3. extract text -> parseJson -> validate(schema)
  4. on parse/validate fail -> repair pass (1x) -> revalidate
  5. on success -> cache.set(cacheKey, result, ttl) -> return
  6. on failure/timeout -> throw AiUnavailable (callers fall back)
```

10.2 Prompt Design

- System prompt states role, hard constraints ('You personalize; you never invent curriculum. Reference only provided day numbers/ids. Output ONLY valid JSON matching the schema.')
- User message carries compact, pre-aggregated context (never raw tables) to control tokens.
- Few-shot: one exemplar input/output per agent embedded in prompts/ files.
- Output contract: a JSON schema per agent; the gateway validates before returning.
- Guardrail line: 'If unsure, prefer keep/no-op over inventing content.'

10.3 Memory & Context

- **No hidden server memory** — Agents are stateless; 'memory' = context we pass in from Supabase (assessment, knowledge_graph, rollups, prior weekly_reports).
- **Context window discipline** — Only summaries/rollups are sent, not full histories; prior review summary (not full text) is included for continuity.
- **Traceability** — Every call logs { agent, cacheKey, model, tokens, latency } to ai_invocations for audit and cost.

10.4 Knowledge Base Boundary

Bedrock receives references to the JSON knowledge base (day numbers, content ids, skills), never authors it. Suggested projects/topics MUST map to existing content; the gateway's validation rejects references that don't exist in the loaded roadmap/assessment JSON.

10.5 Caching

Task	Cache key	TTL
Roadmap overlay	hash(role, level, knowledge_graph)	until assessment retake
Weekly review	user_id + week_index	permanent (row)
Daily motivation	user_id + date	1 day
Project feedback	submission hash	permanent (row)
Recommendation	user_id + date + roadmap_version	1 day

10.6 Retry, Fallback, Latency

- Retries: up to 2 with exponential backoff on throttling/5xx; 1 JSON-repair retry on invalid output.
- Timeout: per-call budget (e.g. 12s). On exceed -> AiUnavailable.
- Fallback per caller: roadmap->raw JSON order; weekly review->lite stats review; motivation->static template; recommendation->next unlocked day; project feedback->rubric-only checklist.
- Latency isolation: all Bedrock work is async/queued; user-visible reads come from Supabase rows.

10.7 Cost Optimization

- Small prompts (rollups only), low max_tokens per agent, aggressive caching by content-stable keys.
- Batch weekly reviews; skip calls when inputs are unchanged (cacheKey hit).
- Track tokens per agent in ai_invocations; alert on anomalies.

10.8 Security

- AWS credentials via server-side env / IAM role only; least-privilege policy limited to InvokeModel on the chosen model.
- No PII beyond what's needed; resume text is summarized to skills before sending, never raw documents.
- Prompt-injection defense: content JSON is data, not instructions; the gateway strips/escapes and the system prompt forbids following embedded instructions.
- All outputs schema-validated before they touch the DB or UI.

10.9 Bedrock Responsibilities (EXHAUSTIVE — nothing else)

- Roadmap Personalization (overlay).
- Weekly Review (narrative + validated roadmap_updates).
- Daily Motivation (one/two lines).
- Project Feedback (rubric-based).
- Learning Recommendations (next best existing action).

Bedrock does NOT: score the assessment, generate questions/days/projects, compute the Talent Score, or make any authoritative data decision. Those are deterministic engine responsibilities.

Implementation Checklist (Mandatory)

Aspect	Implementation Detail
1. Frontend Components	None direct. UI shows Bedrock outputs via 'AI' badges + 'Personalizing...' / 'Generating...' states.
2. Backend Services	lib/ai/bedrockClient (gateway), lib/ai/schemas (per-agent JSON schemas), lib/ai/cache, aiInvocationLogger.
3. Supabase Tables Used	ai_invocations (agent, model, tokens, latency, cache_hit, status), plus per-feature result tables (user_roadmaps, weekly_reports, project_reviews).

Aspect	Implementation Detail
4. APIs Required	Internal only: called by services; not a public endpoint. Optional POST /api/ai/health for smoke checks.
5. Amazon Bedrock Prompt	Central gateway: system+messages+schema+cacheKey; temperature<=0.3; validate+repair; retry+fallback.
6. JSON Files Consumed	content/roadmap/*.json, content/assessment/*.json — referenced (not authored) and used to validate references.
7. State Management	Stateless agents; context passed in; results persisted to Supabase and read by the UI.
8. Error Handling	AiUnavailable => per-caller deterministic fallback; JSON repair pass; schema validation gate.
9. Loading States	Async/queued; UI never blocks; badges reflect pending enrichment.
10. Testing Scenarios	Mock gateway for deterministic tests; malformed-output repair; timeout->fallback; injection resistance; cache-key stability.
11. Future Scalability	Stateless + cached + queued; model id/region configurable; add agents by adding a prompt+schema, not new infra.

Chapter 11 — AI Agents

Seven agents run through the Bedrock gateway (Ch.10). Each is stateless, receives explicit context, returns schema-validated JSON, logs to ai_invocations, and has a deterministic fallback. Note: the Assessment Analysis Agent is ADVISORY only — assessment SCORING remains deterministic (Ch.3).

11.1 Roadmap Agent

Purpose: Produce the personalization overlay (reorder/compress/reinforce) for an assigned roadmap. Never edits content.

Input: knowledge_graph (skills+bands+related_days), roadmap day summaries [{day,title,skills_gained,difficulty}], level.

Output: Overlay JSON { sequence(permutation of 1..45), day_actions{day:{action,reason,badge,extra_focus?}}, focus_skills }.

Prompt (system, abbreviated)

You are the Roadmap Agent. You PERSONALIZE ordering only.
 Rules: reference only provided day numbers; never invent days/content.
 compress = learner already strong; reinforce = weak skill; keep = default.
 sequence MUST be a permutation of the provided day numbers.
 Output ONLY JSON matching the overlay schema.

Memory: Stateless; prior overlay (if any) passed in for continuity.

Tools: None beyond the gateway; validation done by the gateway against roadmap JSON.

Workflow

1. Assemble knowledge_graph + day summaries.
2. Invoke gateway with overlay schema.
3. Validate permutation + known actions + prereq safety.
4. Persist to user_roadmaps.personalization; flip 'personalized' flag.

Failure Recovery: On AiUnavailable/invalid: keep raw JSON day order; retry via next Weekly Review.

Logging: ai_invocations(agent='roadmap_agent', tokens, latency, cache_hit, valid).

Future Improvements: Insert remedial reinforcement days; per-skill micro-sequencing; A/B of ordering strategies.

11.2 Assessment Analysis Agent (Advisory)

Purpose: Turn deterministic skill scores into a human-readable strengths/gaps narrative and study hints. Does NOT change scores or level.

Input: skill_scores[], knowledge_graph, overall_score, level.

Output: { narrative, top_gaps[], quick_wins[], study_focus[] } (all referencing existing skills/days).

Prompt (system, abbreviated)

You are the Assessment Analysis Agent. Scoring is already final.
 Explain results supportively; map gaps to existing roadmap days only.
 Output ONLY JSON per schema. Do not output scores or a level.

Memory: Stateless.

Tools: Gateway only.

Workflow

1. Receive final scores + graph after Finish.
2. Invoke gateway; validate references.
3. Store on `assessment_results.analysis (jsonb)` for the results screen.

Failure Recovery: Fallback: template narrative built from band arrays (weak/strong).

Logging: `ai_invocations(agent='assessment_analysis')`.

Future Improvements: Personalized study time estimates; link to specific videos/articles per gap.

11.3 Daily Mission Agent

Purpose: Add a short motivation line and, on reinforce days, suggest ONE extra practice item from existing content.

Input: current day summary, `day_action`, `streak`, yesterday's completion, `focus_skills`.

Output: { motivation, extra_task? { type, ref, reason } } (ref must exist in the day JSON).

Prompt (system, abbreviated)

You are the Daily Mission Agent. Encourage briefly (≤ 2 sentences).
Only suggest an extra task on reinforce days, referencing existing day content.
Output ONLY JSON per schema.

Memory: Stateless; streak + last-day status passed in.

Tools: Gateway only.

Workflow

1. On mission generation, invoke gateway (`cacheKey=user+date`).
2. Validate `extra_task.ref` against the day content.
3. Attach motivation to the mission header; append `extra_task` if valid.

Failure Recovery: Fallback: static rotating motivation string; no extra task.

Logging: `ai_invocations(agent='daily_mission')`.

Future Improvements: Tone adaptation to user preference; time-of-day aware nudges.

11.4 Weekly Review Agent

Purpose: Generate the weekly narrative + validated `roadmap_updates` (Ch.9).

Input: week stats, weak/strong skills, roadmap position, `missed_topics`, talent delta, prior review summary.

Output: { summary, strengths[], weaknesses[], recommendations[], missed_topics[], suggested_projects[], roadmap_updates{}, motivation }.

Prompt (system, abbreviated)

You are the Weekly Review Agent. Be specific, supportive, and actionable.
`suggested_projects` and `roadmap_updates` MUST reference existing day numbers/projects.
`resequence` (if present) MUST be a valid permutation. Output ONLY JSON per schema.

Memory: Stateless; prior review summary included for continuity.

Tools: Gateway only; gateway validates day/project references + permutation.

Workflow

1. weeklyReviewService assembles input.
2. Invoke gateway; validate roadmap_updates.
3. Persist weekly_reports; merge valid updates into overlay; set last_review_at.

Failure Recovery: Lite deterministic review (stats only), ai_enriched=false; retry enrichment later.

Logging: ai_invocations(agent='weekly_review').

Future Improvements: Trend-aware coaching across weeks; goal renegotiation; sentiment-aware tone.

11.5 Talent Score Recommendation Agent

Purpose: Advise the learner on the highest-leverage actions to raise their Talent Score. Advisory only; never changes the number.

Input: talent_scores breakdown + deltas + weekly goals.

Output: { tips[] { component, action, expected_gain } } referencing real activities.

Prompt (system, abbreviated)

You are the Talent Score Coach. Recommend concrete next actions tied to score components. Never claim to change the score; only suggest actions. Output ONLY JSON per schema.

Memory: Stateless.

Tools: Gateway only.

Workflow

1. On breakdown drawer open (cacheKey=user+date+score_version).
2. Invoke gateway; render tips as cards with 'do it now' links.

Failure Recovery: Fallback: rule-based tips (lowest component => canned advice).

Logging: ai_invocations(agent='talent_reco').

Future Improvements: Simulated 'what-if' score projections; personalized weekly targets.

11.6 Learning Recommendation Agent

Purpose: Pick the single best next action (Next Recommendation on dashboard) from existing unlocked content.

Input: current/next day, weak skills, recent activity, available tasks.

Output: { type, title, ref, reason } referencing an existing day/task.

Prompt (system, abbreviated)

You are the Learning Recommendation Agent. Choose ONE next action from provided options only. Prefer actions that address weak skills without breaking prerequisites. Output ONLY JSON per schema.

Memory: Stateless; resume_skills may be included for cold-start tailoring.

Tools: Gateway only.

Workflow

1. On dashboard build (cacheKey=user+date+roadmap_version).
2. Invoke gateway; validate ref is unlocked & exists.
3. Render Next Recommendation card.

Failure Recovery: Fallback: next unlocked day's first required task.

Logging: `ai_invocations(agent='learning_reco')`.

Future Improvements: Bandit-style optimization on click-through; cross-skill sequencing.

11.7 Project Review Agent

Purpose: Give rubric-based feedback and a 0-100 quality score for a submitted project (feeds the Projects component of Talent Score).

Input: project spec (from day mini_project), submission (repo/link/notes), role, level.

Output: { score(0-100), rubric{ correctness, completeness, code_quality, docs }, feedback[], next_steps[] }.

Prompt (system, abbreviated)

You are the Project Review Agent. Evaluate against the provided rubric only.
Be constructive and specific; cite what to improve. Output ONLY JSON per schema.

Memory: Stateless; project spec + submission passed in.

Tools: Gateway only; (future: read-only repo fetch tool).

Workflow

1. On project submission, assemble spec+submission.
2. Invoke gateway (cacheKey=submission hash); validate score range + rubric keys.
3. Persist project_reviews; trigger talent recompute (Projects component).

Failure Recovery: Fallback: rubric checklist with no numeric score (score deferred); mark ai_enriched=false.

Logging: `ai_invocations(agent='project_review')`.

Future Improvements: Static-analysis tool integration; plagiarism/originality signals; test execution in a sandbox.

Implementation Checklist (Mandatory)

Aspect	Implementation Detail
1. Frontend Components	AiBadge, PersonalizingChip, RecommendationCard, ProjectReviewPanel, TalentTips, AssessmentAnalysisPanel, MotivationBanner.
2. Backend Services	One service per agent wrapping bedrockClient; shared schemas in lib/ai/schemas; agents/ prompt files.
3. Supabase Tables Used	ai_invocations, user_roadmaps, weekly_reports, assessment_results(analysis), project_reviews, talent_scores, daily_tasks.
4. APIs Required	Invoked internally by services (assign, mission, review, dashboard, submit). Optional POST /api/ai/health.
5. Amazon Bedrock Prompt	All seven agents call the Ch.10 gateway with their own system prompt + JSON schema + cacheKey.
6. JSON Files Consumed	content/roadmap/*.json, content/assessment/*.json for reference validation.
7. State Management	Stateless agents; context in, JSON out; results persisted and read by UI.
8. Error Handling	Per-agent deterministic fallback; gateway repair+validate; reference validation drops invalid suggestions.
9. Loading States	Async badges; project review shows 'reviewing...'; recommendation cached daily.
10. Testing Scenarios	Golden-file prompt tests; schema validation; fallback simulation; reference-validation rejects fabricated refs.

Aspect	Implementation Detail
11. Future Scalability	Add an agent = add prompt + schema + service; no infra change; stateless horizontal scale.

Chapter 12 — Supabase Database Design

Postgres on Supabase. All tables use uuid primary keys (default `gen_random_uuid()`), `created_at/updated_at` `timestampz` defaults, and Row Level Security (RLS) so a user can only read/write their own rows. `user_id` columns reference `auth.users(id)` ON DELETE CASCADE. Content tables (roadmaps/assessments) are catalog rows referencing JSON files; content itself stays in the repo.

12.1 Conventions

- PK id uuid default `gen_random_uuid()`; timestamps `timestampz` default `now()`; `updated_at` maintained by a trigger.
- Enums as text + CHECK constraints (portable) unless noted.
- Money/scores as integer; percentages as `numeric(5,2)`.
- Every user-owned table: RLS 'owner can CRUD own rows' + `service_role` bypass for jobs.

12.2 Core Tables

profiles (1:1 with auth.users)

Column	Type	Constraints / Notes
id	uuid	PK = auth.users.id (FK, cascade)
full_name	text	not null, 2-60 chars
display_name	text	nullable
country	text	ISO code
timezone	text	IANA, default 'UTC'
education_level	text	CHECK in set
experience_level	text	CHECK in set
weekly_hours	int	default 10, CHECK 3-40
learning_days	text[]	default '{Mon,Tue,Wed,Thu,Fri}'
role	text	FK roles.key; the chosen career role
github_username	text	nullable
github_connected	bool	default false
linkedin_url	text	nullable
resume_skills	jsonb	extracted skills, nullable
onboarding_step	int	default 1
onboarding_completed	bool	default false, indexed

roles (catalog, seeded)

Column	Type	Constraints / Notes
id	uuid	PK
key	text	unique (genai_engineer, ai_engineer, ai_automation_engineer, data_science_engineer)
label	text	display name
assessment_file	text	content/assessment/... filename
roadmap_beginner_file	text	content/roadmap/... filename
roadmap_intermediate_file	text	content/roadmap/... (note ds misspelling)
skills	jsonb	10 skills for the role

12.3 Assessment Tables

assessments (catalog)

Column	Type	Constraints / Notes
id	uuid	PK
role_key	text	FK roles.key
file	text	assessment JSON filename
version_hash	text	hash of JSON content
total_questions	int	100
total_weight	int	1000

assessment_results

Column	Type	Constraints / Notes
id	uuid	PK
user_id	uuid	FK auth.users, indexed
assessment_id	uuid	FK assessments
attempt_number	int	increments per user+role
seed	text	RNG seed for reproducible selection
question_ids	jsonb	ordered 20 selected ids
status	text	in_progress completed abandoned
overall_score	int	0-100, null until completed
level	text	beginner intermediate, null until completed
analysis	jsonb	advisory narrative (Agent 11.2)
completed_at	timestampz	nullable

skill_scores

Column	Type	Constraints / Notes
id	uuid	PK
assessment_result_id	uuid	FK assessment_results (cascade)
user_id	uuid	FK, indexed
skill	text	one of role's 10 skills
score	int	0-100
band	text	weak developing strong
		UNIQUE(assessment_result_id, skill)

knowledge_graph

Column	Type	Constraints / Notes
id	uuid	PK
assessment_result_id	uuid	FK (cascade)
user_id	uuid	FK, indexed
graph	jsonb	skills[], weak_skills[], strong_skills[], related_days
overall	int	0-100

12.4 Roadmap Tables

roadmaps (catalog)

Column	Type	Constraints / Notes
id	uuid	PK
role_key	text	FK roles.key
level	text	beginner intermediate
file	text	roadmap JSON filename
version_hash	text	hash of JSON
total_days	int	45

user_roadmaps

Column	Type	Constraints / Notes
id	uuid	PK
user_id	uuid	FK, indexed
roadmap_id	uuid	FK roadmaps
role	text	denormalized role key
level	text	beginner intermediate
version	int	personalization overlay version
status	text	active archived completed
personalization	jsonb	overlay {sequence, day_actions, focus_skills}
start_date	date	assignment date
last_review_at	timestamptz	weekly review pointer
		PARTIAL UNIQUE(user_id) WHERE status='active' per role

roadmap_progress

Column	Type	Constraints / Notes
id	uuid	PK
user_roadmap_id	uuid	FK (cascade), indexed
user_id	uuid	FK, indexed
day	int	1-45
status	text	locked unlocked in_progress completed
prereq_days	int[]	resolved prerequisites
unlock_at	timestamptz	nullable
completed_at	timestamptz	nullable
		UNIQUE(user_roadmap_id, day)

12.5 Daily Mission & Task Tables**daily_tasks**

Column	Type	Constraints / Notes
id	uuid	PK
user_id	uuid	FK, indexed
user_roadmap_id	uuid	FK
day	int	roadmap day
date	date	local mission date

Column	Type	Constraints / Notes
type	text	video article quiz assignment practical_task project research commit
ref	text	content id/index within the day
title	text	
est_min	int	estimated minutes
required	bool	counts toward day completion
order_index	int	display order
		UNIQUE(user_id, day, date, type, ref)

task_progress

Column	Type	Constraints / Notes
id	uuid	PK
user_id	uuid	FK, indexed
daily_task_id	uuid	FK (cascade)
status	text	pending completed
xp	int	awarded XP
artifact_url	text	nullable (assignment/project link)
completed_at	timestampz	nullable
		UNIQUE(user_id, daily_task_id) // idempotent completion

quiz_attempts

Column	Type	Constraints / Notes
id	uuid	PK
user_id	uuid	FK, indexed
day	int	roadmap day
score	int	0-100
answers	jsonb	selected indices
passed	bool	

12.6 Projects

project_reviews

Column	Type	Constraints / Notes
id	uuid	PK
user_id	uuid	FK, indexed
day	int	day whose mini_project this is
submission_url	text	repo/link
score	int	0-100, null if fallback
rubric	jsonb	correctness/completeness/quality/docs
feedback	jsonb	feedback[] + next_steps[]
ai_enriched	bool	false if rubric-only fallback

12.7 Tracking, Score & AI Tables

learning_sessions

Column	Type	Constraints / Notes
id	uuid	PK
user_id	uuid	FK, indexed
started_at	timestampz	
ended_at	timestampz	nullable
duration_sec	int	active time only
context	jsonb	day/task refs

talent_scores (append-versioned)

Column	Type	Constraints / Notes
id	uuid	PK
user_id	uuid	FK, indexed
score	int	0-1000
level	text	Explorer..Expert
breakdown	jsonb	8 components
delta	int	vs previous row
created_at	timestampz	indexed (user_id, created_at)

badges

Column	Type	Constraints / Notes
id	uuid	PK
user_id	uuid	FK, indexed
code	text	badge code
earned_at	timestampz	
		UNIQUE(user_id, code)

weekly_goals

Column	Type	Constraints / Notes
id	uuid	PK
user_id	uuid	FK, indexed
week_index	int	
tasks_goal	int	
minutes_goal	int	
tasks_done	int	
minutes	int	
		UNIQUE(user_id, week_index)

weekly_reports

Column	Type	Constraints / Notes
id	uuid	PK
user_id	uuid	FK, indexed
week_index	int	
input	jsonb	assembled stats sent to agent

Column	Type	Constraints / Notes
output	jsonb	review JSON
ai_enriched	bool	false = lite fallback
		UNIQUE(user_id, week_index)

activity_logs (append-only)

Column	Type	Constraints / Notes
id	uuid	PK
user_id	uuid	FK, indexed
action	text	video_watch article_read ... level_up github_commit
target	text	day/task/ref
meta	jsonb	arbitrary
date	date	indexed (user_id, date)

progress_rollups (nightly materialized)

Column	Type	Constraints / Notes
id	uuid	PK
user_id	uuid	FK, indexed
period	text	day week month
period_key	text	e.g. 2026-W30
metrics	jsonb	tasks/minutes/active_days/quiz_avg
		UNIQUE(user_id, period, period_key)

ai_invocations (audit/cost)

Column	Type	Constraints / Notes
id	uuid	PK
user_id	uuid	FK, indexed, nullable
agent	text	roadmap_agent weekly_review ...
model	text	BEDROCK_MODEL_ID
tokens_in	int	
tokens_out	int	
latency_ms	int	
cache_hit	bool	
status	text	ok fallback error

12.8 Relationships (summary)

- auth.users 1—1 profiles; profiles.role -> roles.key.
- assessment_results 1—* skill_scores, 1—1 knowledge_graph.
- user_roadmaps 1—* roadmap_progress; user_roadmaps.roadmap_id -> roadmaps.
- daily_tasks 1—1 task_progress; daily_tasks -> user_roadmaps.
- user_id fans out to talent_scores, badges, weekly_goals, weekly_reports, learning_sessions, activity_logs, project_reviews, ai_invocations.

12.9 Indexes

```

CREATE INDEX ON assessment_results(user_id, status);
CREATE INDEX ON roadmap_progress(user_roadmap_id, status);
CREATE INDEX ON daily_tasks(user_id, date);
CREATE INDEX ON task_progress(user_id, completed_at);
CREATE INDEX ON learning_sessions(user_id, started_at);
CREATE INDEX ON talent_scores(user_id, created_at DESC);
CREATE INDEX ON activity_logs(user_id, date);
CREATE UNIQUE INDEX one_active_roadmap ON user_roadmaps(user_id, role) WHERE status='active';

```

12.10 Triggers

- set_updated_at BEFORE UPDATE on every table -> now().
- on task_progress INSERT/UPDATE (completed) -> recompute day completion + enqueue talent recompute.
- on roadmap_progress UPDATE status='completed' -> evaluate dependents' unlock.
- on assessment_results UPDATE status='completed' -> assign roadmap (via edge function / service).
- on talent_scores INSERT -> emit level_up activity_log if band changed.

12.11 RLS Policies

```

ALTER TABLE <t> ENABLE ROW LEVEL SECURITY;
-- owner read/write
CREATE POLICY own_sel ON <t> FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY own_ins ON <t> FOR INSERT WITH CHECK (auth.uid() = user_id);
CREATE POLICY own_upd ON <t> FOR UPDATE USING (auth.uid() = user_id);
-- catalog tables (roles, roadmaps, assessments): read-only to authenticated
CREATE POLICY cat_sel ON roles FOR SELECT USING (auth.role() = 'authenticated');
-- background jobs use the service_role key (bypasses RLS)

```

Implementation Checklist (Mandatory)

Aspect	Implementation Detail
1. Frontend Components	None (schema). Types generated via `supabase gen types typescript` into types/database.ts.
2. Backend Services	Migrations in supabase/migrations/*.sql; seed roles/roadmaps/assessments from repo file listing.
3. Supabase Tables Used	All tables in this chapter.
4. APIs Required	Data accessed via services using the typed Supabase client (server) with RLS; jobs use service_role.
5. Amazon Bedrock Prompt	ai_invocations stores every Bedrock call; no schema authored by Bedrock.
6. JSON Files Consumed	Catalog tables (roles/roadmaps/assessments) reference content/*.json filenames + version_hash.
7. State Management	Server components read via RLS-scoped client; mutations via Server Actions/route handlers.
8. Error Handling	CHECK constraints + UNIQUE keys enforce invariants; idempotency via natural unique keys.
9. Loading States	N/A (DB). Query timeouts + retries in the data layer.
10. Testing Scenarios	Migration up/down; RLS policy tests (user A cannot read user B); constraint tests; trigger tests.
11. Future Scalability	Append-versioned score/logs + nightly rollups; partial unique for single active roadmap; indexes on (user_id, date).

Chapter 13 — API Design

APIs are Next.js 15 Route Handlers under `app/api/**` (Node runtime) plus Server Actions for form-style mutations. All are authenticated via Supabase Auth (the session cookie); the handler derives `auth.uid()` server-side and RLS enforces ownership. Every response uses a typed envelope `{ data, error }` and Zod-validated inputs.

13.1 Conventions

- Auth: every route calls `createServerClient()`; 401 if no session. `user_id` is NEVER taken from the body.
- Validation: Zod schema per route; invalid -> 422 with field errors.
- Envelope: 2xx -> `{ data }`, error -> `{ error: { code, message, details? } }`.
- Idempotency: mutation routes use natural keys; safe to retry.
- Rate limits: sensitive routes (assessment retake, ai) limited per user.

13.2 Endpoint Catalog

Method & Path	Purpose	Auth
POST <code>/api/onboarding</code>	Upsert a wizard step (idempotent by step)	user
POST <code>/api/onboarding/github</code>	Link GitHub handle/identity	user
POST <code>/api/onboarding/resume</code>	Upload+parse resume -> <code>resume_skills</code>	user
POST <code>/api/assessment/start</code>	Create attempt, select 20 Qs (seeded)	user
POST <code>/api/assessment/answer</code>	Record answer, return reveal	user
POST <code>/api/assessment/submit</code>	Finish, score, build graph, set level	user
GET <code>/api/assessment/history</code>	List attempts + per-skill trend	user
POST <code>/api/roadmap/assign</code>	Select file, expand days, queue overlay	user
GET <code>/api/roadmap/:id</code>	Get user roadmap + progress + overlay	user
POST <code>/api/roadmap/overlay/rebuild</code>	Re-run Roadmap Agent	user
GET <code>/api/dashboard</code>	Aggregate dashboard payload	user
GET <code>/api/mission/today</code>	Get/generate today's mission	user
POST <code>/api/tasks/:id/complete</code>	Complete a task, award XP	user
POST <code>/api/mission/rollover</code>	Roll incomplete tasks forward	user
POST <code>/api/quiz/:day/submit</code>	Submit quiz attempt	user
POST <code>/api/projects/submit</code>	Submit project -> Project Review Agent	user
POST <code>/api/session/heartbeat</code>	Session keep-alive (active time)	user
POST <code>/api/session/close</code>	Close session	user
GET <code>/api/progress/weekly</code>	Weekly rollup	user
GET <code>/api/progress/monthly</code>	Monthly rollup	user
GET <code>/api/talent/current</code>	Latest talent score + breakdown	user
GET <code>/api/talent/history</code>	Talent trend	user
GET <code>/api/badges</code>	Earned badges	user
POST <code>/api/weekly-review/run</code>	Run/generate weekly review	user
GET <code>/api/weekly-review/latest</code>	Latest review	user
GET <code>/api/weekly-review/history</code>	Review history	user
POST <code>/api/github/sync</code>	Internal: daily GitHub sync	service
POST <code>/api/talent/recompute</code>	Internal: recompute score	service

13.3 Detailed Examples

POST /api/assessment/answer

```
// Request
{ "attempt_id": "uuid", "question_id": "Q7", "selected_index": 2 }
// 200 Response
{ "data": { "correct_answer": 0, "is_correct": false,
            "explanation": "...", "answered": 7, "total": 20 } }
// 422 (bad index)
{ "error": { "code": "VALIDATION", "message": "selected_index out of range" } }
```

POST /api/assessment/submit

```
// Request
{ "attempt_id": "uuid" }
// 200 Response
{ "data": { "overall_score": 58, "level": "beginner",
            "skills": [{ "skill": "Python", "score": 82, "band": "strong" }, ...],
            "next": "/api/roadmap/assign" } }
// 409 if not all questions answered
{ "error": { "code": "INCOMPLETE", "message": "20 answers required" } }
```

POST /api/tasks/:id/complete

```
// Request
{ "artifact_url": "https://github.com/..." } // optional
// 200 Response
{ "data": { "task_id": "uuid", "xp_awarded": 30, "day_completed": false,
            "streak": 5, "talent_delta": +4 } }
```

13.4 Error Codes

HTTP	code	Meaning
400	BAD_REQUEST	Malformed JSON / missing body
401	UNAUTHENTICATED	No/invalid session
403	FORBIDDEN	RLS/ownership violation
404	NOT_FOUND	Resource missing
409	CONFLICT	State conflict (e.g. incomplete submit, cooldown)
422	VALIDATION	Zod field errors (details[])
429	RATE_LIMITED	Too many attempts (retake/AI)
503	AI_UNAVAILABLE	Bedrock down -> fallback used
500	INTERNAL	Unexpected server error

13.5 Validation & Security

- Zod schemas colocated in each route; shared schemas in lib/validation.
- No user_id/role from client is trusted; always server-derived.
- Assessment answer/submit are server-authoritative (client can't set scores).
- Internal routes (github/sync, recompute) require service role / signed job token, not user sessions.
- CORS locked to the app origin; mutations require same-site cookies.

Implementation Checklist (Mandatory)

Aspect	Implementation Detail
1. Frontend Components	API client wrappers in lib/api/*; TanStack Query hooks (useMission, useDashboard, useSubmitAssessment, ...).

Aspect	Implementation Detail
2. Backend Services	Route Handlers under app/api/**; Server Actions for wizard; services do the work, routes stay thin.
3. Supabase Tables Used	All (each route maps to its chapter's tables).
4. APIs Required	This catalog is the contract; keep in sync with lib/api types.
5. Amazon Bedrock Prompt	assign/overlay, weekly-review/run, projects/submit, dashboard(recommendation) invoke agents via services.
6. JSON Files Consumed	Assessment/roadmap files read by assessment/roadmap/mission routes.
7. State Management	Server-authoritative; client caches via TanStack Query with envelope-aware error handling.
8. Error Handling	Uniform { error:{code,message,details} }; typed error map above.
9. Loading States	Hooks expose isLoading/isPending; routes stream where useful (dashboard).
10. Testing Scenarios	Contract tests per route (happy + each error code); auth/RLS negative tests; idempotency retries.
11. Future Scalability	Stateless handlers; heavy work queued; rate limits protect AI/retake paths.

Chapter 14 — Folder Structure

Next.js 15 App Router. The Personalized Learning Engine lives in a route group `app/(learning)` plus shared `lib/services/agents/prompts`. The existing `content/` directory (roadmap + assessment JSON) is consumed read-only and must NOT be renamed.

14.1 Complete Tree

```

app/
  (learning)/
    onboarding/          page.tsx (OnboardingWizard)
    assessment/          page.tsx (AssessmentRunner)
    assessment/history/  page.tsx
    roadmap/assign/      page.tsx (AssigningLoader)
    dashboard/          page.tsx (DashboardHome)
    mission/            page.tsx
    progress/           page.tsx
    talent/             page.tsx
    weekly-review/      page.tsx
  api/
    onboarding/route.ts  onboarding/github/route.ts  onboarding/resume/route.ts
    assessment/start/route.ts  answer/route.ts  submit/route.ts  history/route.ts
    roadmap/assign/route.ts  [id]/route.ts  overlay/rebuild/route.ts
    dashboard/route.ts  mission/today/route.ts  mission/rollover/route.ts
    tasks/[id]/complete/route.ts  quiz/[day]/submit/route.ts  projects/submit/route.ts
    session/heartbeat/route.ts  session/close/route.ts
    progress/weekly/route.ts  monthly/route.ts
    talent/current/route.ts  history/route.ts  recompute/route.ts  badges/route.ts
    weekly-review/run/route.ts  latest/route.ts  history/route.ts
    github/sync/route.ts
  components/
    onboarding/  assessment/  dashboard/  mission/  roadmap/  progress/  talent/
    weekly-review/  ai/ (AiBadge, PersonalizingChip)  ui/ (shared primitives)
  hooks/
    useMission.ts  useDashboard.ts  useAssessment.ts  useTalent.ts  useWeeklyReview.ts
  lib/
    supabase/ (server.ts, client.ts, service.ts)
    services/ (onboarding, assessment, roadmap, mission, progress,
              talentScore, weeklyReview, dashboard, github, session)
    ai/ (bedrockClient.ts, cache.ts, schemas/, invocationLogger.ts)
    agents/ (roadmapAgent.ts, assessmentAnalysisAgent.ts, dailyMissionAgent.ts,
            weeklyReviewAgent.ts, talentRecoAgent.ts, learningRecoAgent.ts,
            projectReviewAgent.ts)
    prompts/ (roadmap.md, weekly_review.md, daily_motivation.md,
            project_review.md, learning_reco.md, assessment_analysis.md,
            talent_reco.md)
    content/ (loader.ts, cache.ts) // reads ../content JSON
    scoring/ (scoringEngine.ts, graphBuilder.ts, levelClassifier.ts)
    validation/ (zod schemas)  utils/ (date, tz, parse)
  content/ // EXISTING, read-only
    assessment/ *_beginner_assessment.json (4 files)
    roadmap/    roadmap_*_{beginner,intermediate}.json (8 files)
  types/ (database.ts generated, domain.ts, api.ts)
  supabase/ migrations/*.sql  seed.sql

```

14.2 Layering Rules

- **Route handlers** — Thin: auth + validate + call a service + envelope. No business logic.
- **Services** — All business logic; the only layer that touches Supabase + content loader + agents.
- **Agents** — Wrap bedrockClient with a prompt (prompts/) + schema (ai/schemas). No DB writes themselves; return validated JSON to services.

- **Content loader** — Single module reads/validates/caches content/*.json by role+level+hash. Nothing else reads files directly.
- **Scoring** — Pure functions, no I/O; unit-tested for determinism.

14.3 Naming & Conventions

- Role keys are the enum `genai_engineer | ai_engineer | ai_automation_engineer | data_science_engineer`.
- Content filenames are constants in `lib/content/loader.ts` (incl. `roadmap_ds_intermediate.json`).
- Server-only modules import `'server-only'`; never leak the `service_role` key to client bundles.
- Components are PascalCase; hooks camelCase with `use-` prefix; services camelCase objects.

Implementation Checklist (Mandatory)

Aspect	Implementation Detail
1. Frontend Components	All under <code>components/**</code> grouped by feature + <code>hooks/**</code> ; pages in <code>app/(learning)/**</code> .
2. Backend Services	<code>lib/services/**</code> as the business layer; <code>app/api/**</code> thin handlers; <code>lib/agents</code> + <code>lib/ai</code> for Bedrock.
3. Supabase Tables Used	Accessed only through <code>lib/supabase</code> clients inside services.
4. APIs Required	Mirror the <code>app/api</code> tree in Ch.13.
5. Amazon Bedrock Prompt	<code>lib/ai/bedrockClient</code> + <code>lib/agents/*</code> + <code>lib/prompts/*</code> .
6. JSON Files Consumed	<code>content/assessment/*</code> , <code>content/roadmap/*</code> via <code>lib/content/loader.ts</code> only.
7. State Management	<code>hooks/**</code> (TanStack Query) + Zustand stores in <code>components/onboarding</code> .
8. Error Handling	<code>lib/validation</code> + a shared error envelope util used by all routes.
9. Loading States	Route-level <code>loading.tsx</code> + component skeletons in <code>components/ui</code> .
10. Testing Scenarios	<code>__tests__</code> colocated; <code>e2e</code> in <code>e2e/</code> ; scoring + agents have dedicated unit suites.
11. Future Scalability	Feature-grouped structure lets new roles/agents be added without cross-cutting changes.

Chapter 15 — JSON Standards

These are the exact contracts the engine consumes. Assessment and Roadmap are top-level files in content/. Projects, Quiz, Articles, Assignments, Research Papers and Videos are sub-structures EMBEDDED inside each roadmap day (the engine derives task types from them). Knowledge Graph is engine-generated and stored in Supabase. Every file is validated on load; a schema violation fails closed.

15.1 Assessment JSON

```
{ assessment: {
  id:str, title:str, role:str, difficulty:'Beginner',
  type:'adaptive_placement_test', provider:'CalibiAI',
  total_questions:100, total_weight:1000, passing_score:null,
  question_bank:true, questions_per_skill:10,
  selection:{ strategy:'random', questions_per_skill_selected:2,
    generated_assessment_size:20 },
  difficulty_distribution:{ easy:70, medium:30 },
  difficulty_per_skill:{ easy:7, medium:3 },
  skills_assessed:[str × 10], answer_reveal:'on_select' },
  questions:[ { id:str, topic:str, skill:str,
    difficulty:'Easy'|'Medium', weight:10, question:str,
    options:[str × 4], correct_answer:int(0..3),
    explanation:str, roadmap_reference:str, estimated_time:str } × 100 ] }
```

Invariants: 100 questions = 10 skills × 10; 7 Easy + 3 Medium per skill; correct_answer is 0-based; answers/explanations are server-gated (Ch.3).

15.2 Roadmap JSON

```
{ roadmap:{ title:str, role:str, level:'Beginner'|'Intermediate',
  total_days:45, provider:'CalibiAI', description:str, outcome:str },
  days:[ { day:int(1..45), title:str, objectives:[str], topics:[str],
    estimated_time:str, difficulty:str, prerequisites:[str],
    beginner_explanation|intermediate_explanation:str,
    practical_task:str, mini_project:str, assignment:str,
    quiz:{...}, youtube:[...], official_docs:[...],
    github_repositories:[...], research_papers:[...],
    skills_gained:[str], expected_outcome:str } × 45 ] }
```

15.3 Embedded Sub-Schemas (inside a day)

Quiz JSON (day.quiz)

```
{ questions:[ { question:str, options:[str], correct_answer:int(0-based),
  explanation:str } ], pass_score:int }
```

Videos JSON (day.youtube[])

```
{ title:str, url:str, channel?:str, duration_min?:int }
```

Articles / Official Docs (day.official_docs[])

```
{ title:str, url:str, source?:str, est_min?:int }
```

Assignment (day.assignment)

```
// string prompt; the engine wraps it as an assignment task with optional artifact_url
```

Project (day.mini_project)

```
// string spec; engine creates a project task + Project Review lifecycle.
// Review rubric keys: correctness, completeness, code_quality, docs (0-100 each).
```

GitHub Repositories (day.github_repositories[])

```
{ name:str, url:str, purpose?:str }
```

Research Papers (day.research_papers[])

```
{ title:str, url:str, authors?:str, year?:int } // optional/bonus task
```

15.4 Knowledge Graph JSON (engine-generated)

```
{ role:str, level:str, overall:int(0..100),
  skills:[ { skill:str, score:int, band:'weak'|'developing'|'strong',
             related_days:[int], recommendation:'compress'|'reinforce'|'keep' } ],
  weak_skills:[str], strong_skills:[str], generated_at:ISO8601 }
```

15.5 Personalization Overlay JSON (engine + Roadmap Agent)

```
{ sequence:[int] // permutation of day numbers,
  day_actions:{ '<day>': { action:'compress'|'reinforce'|'keep',
                           reason:str, badge?:str, extra_focus?:[str] } },
  focus_skills:[str], version:int, generated_by:str, generated_at:ISO8601 }
```

15.6 Validation Rules (load-time)

- Assessment: exactly 100 questions; per-skill counts 10 (7E/3M); correct_answer in 0..3; 4 options.
- Roadmap: days 1..45 contiguous; each day has all required fields; exactly one of beginner_/intermediate_explanation matching the file level.
- Cross-check: role in file matches the roles catalog; skills_assessed length 10.
- On any violation: throw SchemaError, log to activity_logs, block the dependent flow (no silent partial data).
- Validators implemented with Zod in lib/validation and run inside lib/content/loader.ts.

Implementation Checklist (Mandatory)

Aspect	Implementation Detail
1. Frontend Components	None; UI trusts validated data. Type defs in types/domain.ts generated from these schemas.
2. Backend Services	lib/content/loader.ts validates + caches; lib/validation holds Zod schemas mirroring this chapter.
3. Supabase Tables Used	Catalog tables store filename + version_hash; generated JSON (graph/overlay) stored in Supabase jsonb.
4. APIs Required	assessment/roadmap/mission routes rely on these contracts.
5. Amazon Bedrock Prompt	Agent outputs (overlay, review, project rubric) validated against these schemas before persistence.
6. JSON Files Consumed	content/assessment/*.json, content/roadmap/*.json + embedded sub-schemas.
7. State Management	Loaded once per version; cached in-memory/edge by role+level+hash.
8. Error Handling	SchemaError fails closed; never render unvalidated content or persist unvalidated AI output.
9. Loading States	Content load is fast + cached; first load per version validates.
10. Testing Scenarios	Contract tests parse every real repo file against these Zod schemas in CI (all 4 assessments + 8 roadmaps).
11. Future Scalability	Adding roles/levels = adding files that satisfy these schemas; validators catch drift automatically.

Chapter 16 — Complete Sequence Diagram

End-to-end flow across Frontend -> API -> Supabase -> Bedrock -> Database -> Dashboard -> Tracking -> Weekly Review.
Deterministic steps are synchronous; Bedrock steps are async and never block the UI.

16.1 Login -> Roadmap (first run)



16.2 Daily Learning Loop

```
User -> GET /api/mission/today -> missionSvc.generateForDay()
      reads roadmap day JSON + overlay -> writes daily_tasks
      (async) Daily Mission Agent -> motivation / extra task
User -> completes task -> POST /api/tasks/:id/complete
      task_progress upsert -> XP -> activity_logs
      if day complete -> roadmap_progress=completed -> unlock dependents
      enqueue talentScoreSvc.recompute() -> talent_scores (+delta, level, badges)
Session: heartbeat/close -> learning_sessions (active minutes)
Dashboard cache invalidated -> next read reflects new XP/streak/score
```

16.3 Weekly Review Loop

```
Scheduler (per tz) -> eligible users (>=7 active days)
weeklyReviewSvc.assembleInput() <- rollups + skills + talent delta
-> Weekly Review Agent (Bedrock) -> strict-JSON review + roadmap_updates
validate refs/permutation -> merge into user_roadmaps.personalization
persist weekly_reports -> set last_review_at -> regenerate today's mission if changed
User -> GET /weekly-review/latest -> reads the row (no Bedrock on read)
Fallback: Bedrock down -> lite stats review (ai_enriched=false), retry later
```

16.4 Failure & Fallback Paths

Step	Failure	Fallback
Roadmap overlay	Bedrock down/invalid	Raw JSON day order; retry at weekly review
Weekly review	Bedrock down/invalid	Lite deterministic review

Step	Failure	Fallback
Daily motivation	Bedrock down	Static rotating line; no extra task
Project review	Bedrock down	Rubric checklist, score deferred
Recommendation	Bedrock down	Next unlocked day's first task
Content load	Schema violation	Fail closed + log; block dependent flow

16.5 End-to-End Invariants

- The UI never blocks on Bedrock; deterministic data is always available first.
- Scores, level, unlocking and Talent Score are reproducible from Supabase alone.
- Content JSON is never mutated; personalization is overlay-only and reversible.
- Every write is idempotent by a natural key; retries are safe.
- RLS guarantees users only ever touch their own rows.

Implementation Checklist (Mandatory)

Aspect	Implementation Detail
1. Frontend Components	Full flow across OnboardingWizard -> AssessmentRunner -> AssigningLoader -> DashboardHome -> Mission/Progress/Talent/WeeklyReview.
2. Backend Services	onboarding/assessment/roadmap/mission/progress/talentScore/weeklyReview/dashboard/session services + scheduler.
3. Supabase Tables Used	All tables from Ch.12 participate; see per-step notes.
4. APIs Required	All routes from Ch.13 in sequence.
5. Amazon Bedrock Prompt	Async overlay, weekly review, motivation, project review, recommendation — each with fallback.
6. JSON Files Consumed	content/assessment/*.json (placement), content/roadmap/*.json (days/tasks).
7. State Management	Server-authoritative writes; TanStack Query reads; optimistic task completion.
8. Error Handling	Per-step fallback table above; content schema fails closed.
9. Loading States	Assigning + Personalizing + Generating states; dashboard streams.
10. Testing Scenarios	One full E2E test from login to first weekly review; chaos test with Bedrock disabled proving all fallbacks.
11. Future Scalability	Async queues + caches + rollups keep the synchronous path O(1); agents stateless.

16.6 Build Order for the Agent

1. Migrations + RLS + seed roles/roadmaps/assessments (Ch.12).
2. lib/content/loader + Zod schemas + scoring engine (Ch.15, Ch.3).
3. Onboarding wizard + routes (Ch.2).
4. Assessment runner + scoring + graph + level (Ch.3).
5. Roadmap assign + expand + unlock (Ch.4), raw order first.
6. Dashboard aggregate + Today's Mission + task completion (Ch.5, Ch.6).
7. Progress tracking + sessions + rollups (Ch.7).
8. Talent Score + badges + levels (Ch.8).
9. Bedrock gateway + agents, wired as async overlays/fallbacks (Ch.10, Ch.11).
10. Weekly Review engine + scheduler (Ch.9).
11. E2E + chaos (Bedrock-off) tests (Ch.16).